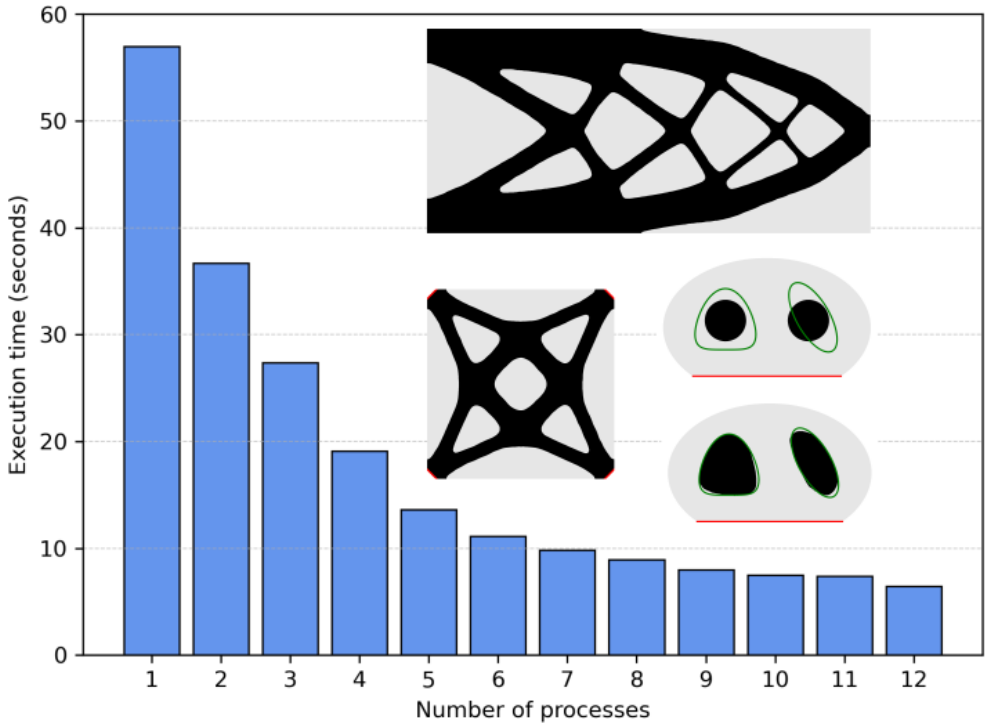
Graphical Abstract

FormOpt: A FEniCSx toolbox for level set-based shape optimization supporting parallel computing

Josué D. Díaz-Avalos, Antoine Laurain



Highlights

FormOpt: A FEniCSx toolbox for level set-based shape optimization supporting parallel computing

Josué D. Díaz-Avalos, Antoine Laurain

- FEniCSx toolbox for 2D and 3D shape and topology optimization based on a level-set method and the distributed shape derivative.
- Introduction to the distributed shape derivative and its tensor representation.
- Detailed description of the parallel FEniCSx implementation.
- Support for three parallelization paradigms: data parallelism, task parallelism, and mixed parallelism combining both approaches.
- Proximal-perturbed Lagrangian method for imposing geometric equality constraints.
- Broad set of numerical examples with linear and nonlinear PDE constraints, including inverse problems, structural mechanics, heat conduction, and population dynamics, illustrating the flexibility of the framework.

FormOpt: A FEniCSx toolbox for level set-based shape optimization supporting parallel computing

Josué D. Díaz-Avalos[a,*], Antoine Laurain[a]

[a]*Fakultät für Mathematik, Universität Duisburg-Essen, Thea-Leymann-Straße 9, Essen, 45127, , Germany*

Abstract

This article presents the toolbox `FormOpt` for two- and three-dimensional shape optimization with parallel computing capabilities, built on the `FEniCSx` software framework. We introduce fundamental concepts of shape sensitivity analysis and their numerical applications, mainly for educational purposes, while also emphasizing computational efficiency via parallelism for practitioners. We adopt an optimize-then-discretize strategy based on the distributed shape derivative and its tensor representation, following the approach presented in "A level set-based structural optimization code using FEniCS" (Laurain, 2018) and extending it in several directions. The numerical shape modeling relies on a level set method, whose evolution is driven by a descent direction computed from the shape derivative. Geometric constraints are treated accurately through a Proximal-Perturbed Lagrangian approach. `FormOpt` leverages the powerful features of `FEniCSx`, particularly its support for weak formulations of partial differential equations, diverse finite element types, and scalable parallelism. The implementation supports three different parallel computing modes: data parallelism, task parallelism, and a mixed mode. Data parallelism exploits `FEniCSx`'s mesh partitioning features, and we implement a task parallelism mode which is useful for problems governed by a set of partial differential equations with varying parameters. The mixed mode conveniently combines both strategies to achieve efficient utilization of computational resources.

Keywords: shape optimization, distributed shape derivative, level set method, FEniCSx, parallel computing

1. Introduction

Shape and topology optimization [1, 2] has built over the years a solid theoretical basis and has found its way in many industrial applications for optimal design thanks to the fast development of computational capacities. The topic has found natural applications first in structural and fluid mechanics, then has been developed for various other partial differential equations (PDEs) constraints.

Different approaches have been developed for shape and topology optimization. The solid isotropic microstructure with penalty (SIMP) method [3], based on a material density approach, is the most popular in structural mechanics and is the basis of many educational codes since the work [4]. It has been extended to different frameworks, beyond structural optimization. Another widely used approach is the level-set paradigm, with different variations of the level-set method introduced by Osher and Sethian in [5]. Other important methods include phase-field [6] and topological derivative-based approaches [7]. Reference works for the level set approach in structural optimization are [8, 9]. Given the vast literature on shape and topology optimization, we provide only a few illustrative references here and refer the reader to review articles for a more comprehensive overview, see for instance [10, 11].

The range of software available for shape and topology optimization is rapidly expanding. Along commercial softwares, numerous open-source tools have been developed. For topology optimization in structural mechanics, the trend of educational codes began in 2001 with Sigmund's 99-line `MATLAB` code [4], followed by an improved version in [12]. Another pioneering contribution is the `FreeFEM` implementation [13]. Since then, codes have been released for various platforms, including `NGsolve` [14], `FEniCS` [15, 16, 17] and `Firedrake` [18]. An open-source `Python` implementation of the Null Space Optimizer is also available in [19]. For a comparison of algorithms, we refer to [10].

*Corresponding author

Email addresses: `josue.diazavalos@uni-due.de` (Josué D. Díaz-Avalos), `antoine.laurain@uni-due.de` (Antoine Laurain)

A significant portion of the literature and educational implementations in structural and topology optimization is based on the SIMP method [4]. Another widely used class of methods is the level-set approach, which exists in several variants, as reviewed in [11]. Some formulations employ a Heaviside or smoothed Heaviside function to represent the domain [20], while others are built upon the concept of the topological derivative [7, 21, 22]. A further line of level set-based methods relies on shape sensitivity analysis in the infinite-dimensional setting, using shape derivatives [1, 2]. Educational codes following this approach include [23, 24]. These methods typically employ the boundary representation of the shape derivative, known as the Hadamard form, which must then be extended into the domain for use within a level-set framework [25]. In the present work, we base our approach on the so-called *distributed* or *weak shape derivative* [26, 27]. This formulation offers several advantages including higher accuracy [28, 29], weaker regularity requirements [26], and a natural framework for domain extension. An educational code for structural mechanics based on this approach was first proposed in [17]. The present work represents a natural continuation and extension of [17], incorporating additional and modernized features. Our overall objective is to provide a framework for learning the fundamental principles of shape sensitivity analysis and their numerical implementation, with an educational focus. The `FEniCSx` platform [15] employs the Unified Form Language (UFL) to represent weak formulations of PDEs, offering an intuitive notation that closely mirrors the underlying mathematics. At the same time, we place strong emphasis on computational efficiency by leveraging the parallel capabilities of `FEniCSx`, enabling fast and practical testing of shape optimization problems. A distinguishing feature of our approach is its emphasis on the optimize-then-discretize paradigm: the shape derivative is derived in the infinite-dimensional setting, and a discrete version is then used in the numerical algorithm.

With the steady increase in computational power, parallel computing has become not only commonplace but often essential for tackling large-scale problems. This trend is also visible in the topology optimization community, where several recent educational codes incorporate parallel capabilities. Examples include a `Julia`-based toolbox [30], a parallel `FreeFEM` framework [22], a PETSc framework [31], and a `FEniCSx` implementation for 2D and 3D topology optimization [32]. The latest version of `cashocs`, built on FEniCS, has also introduced MPI-based parallelism [16, 33]. In this work, we propose a natural extension of the distributed shape derivative approach introduced in [17] by incorporating parallel computing capabilities. Unlike many educational papers on topology optimization, which focus primarily on structural mechanics, our aim is to provide a broader perspective by addressing shape optimization problems subject to different types of PDE constraints. A particularly relevant class of problems considered here are inverse problems, where task parallelism plays a key role, since these problems typically involve families of PDE constraints with varying parameters [34, 35, 36]. Standard applications in structural mechanics and linear elasticity are also included as a particular case.

We extend the work of [17] in several directions. First, in the present framework we rely exclusively on finite elements, whereas [17] combined finite elements for solving the PDE constraints with finite differences for solving the Hamilton–Jacobi equation governing domain evolution. This unified finite element approach enables more flexible geometries and makes more efficient use of `FEniCSx` resources. Second, while [17] focused exclusively on linear elasticity, we address more general PDE constraints and illustrate the framework through a broader set of examples. We also provide a systematic way of handling volume constraints without resorting to penalization in the cost functional. Another major improvement is the integration of parallel computing, which allows for substantial speedups in numerical experiments. Three paradigms of parallelism are supported: data parallelism, distributing computations across multiple processes; task parallelism, enabling for example the simultaneous solution of the same PDE with different sources and mixed parallelism, combining both approaches. An additional objective of this paper is to offer an accessible introduction to parallelization and its application to PDE-constrained shape optimization. The foundation of our framework remains the tensor representation of the distributed shape derivative, as in [17]. Practically, this means that when considering a new problem, the main task is to specify the appropriate PDE and geometry, and to analytically compute the corresponding tensors of the distributed shape derivative. The `FormOpt` files are available for download at `github.com/JD26/FormOpt`, along with several tutorials.

The structure of the paper is as follows. In Section 2, the model shape optimization with constraints is presented, and the basic notions of distributed and boundary expressions of the shape derivative, as well as the tensor representation are introduced. In Section 3, an inverse problem for linear elasticity is presented, which allows us to explain general principles of shape derivative computation through a concrete example. In Section 4, we analyze the error arising from the domain approximation employed in the numerical method. In Section 5, the main algorithm employed in `FormOpt` is described. In Section 6, the main components

of the numerical implementation are discussed in details. In particular, the discretization of the level set equation and the parallelism paradigms are explained. In Section 7, numerical results are given for several model problems, including benchmarks of compliance minimization for linear elasticity. Several performance tests are included, among them an evaluation of parallel scalability. In addition, a concise comparison of toolboxes for shape and topology optimization is presented. Section 8 concludes the work and outlines potential directions for future research. Finally, an appendix is provided with additional implementation details and selected code snippets.

2. Model problem and shape derivatives

Let $\mathcal{D} \subset \mathbb{R}^d$ be an open, bounded domain with boundary $\Gamma \coloneqq \partial\mathcal{D}$. Throughout the paper, $\mathcal{D}$ is fixed, while $\Omega \subset \overline{\mathcal{D}}$ represents the variable domain subject to optimization. The spaces $H^1(\mathcal{D})$ and $H^1_0(\mathcal{D})$ denote the usual Sobolev spaces. The notation $\chi_\Omega \colon \overline{\mathcal{D}} \to \mathbb{R}$ stands for the indicator function of Ω. The notation n is used for both the outward unit normal vector to $\mathcal{D}$ and to Ω. Let I denote the d-dimensional identity matrix and $|\cdot|_\infty$ the infinity norm. Given a vector-valued function $w \in H^1(\mathcal{D})^d$, Dw denotes its Jacobian matrix. We use $\mathbf{0}$ for the zero vector of $\mathbb{R}^d$ and $\mathbf{1}$ for the all-ones vector of $\mathbb{R}^{N_c}$, where N_c is the number of constraints. We denote by S^+ and S^- the restrictions of a function S defined on $\mathcal{D}$ to Ω and $\mathcal{D} \setminus \overline{\Omega}$, respectively.

We consider the following generic shape optimization problem with constraints:

$$\min_{\Omega \subset \mathcal{D}} \mathcal{J}(u, \Omega) \quad \text{subject to} \quad C(\Omega) = \mathbf{1},\ e(u, \Omega) = 0, \tag{1}$$

where $\mathcal{J} \colon \mathbb{H} \times \mathbb{P} \to \mathbb{R}$ is a cost functional, $C \colon \mathbb{P} \to \mathbb{R}^{N_c}$ is a vector-valued constraint function, $e \colon \mathbb{H} \times \mathbb{P} \to \mathbb{H}^*$ is a PDE constraint for u, $\mathbb{P}$ is a set of subsets of $\mathcal{D}$, $\mathbb{H}$ is an appropriate function space, and $\mathbb{H}^*$ is its dual space. The additional constraint $C(\Omega) = \mathbf{1}$ is typically a geometric constraint, such as a volume or perimeter constraint. Assuming $u(\Omega)$ is the unique solution of $e(u, \Omega) = 0$, we introduce the reduced cost functional $J(\Omega) \coloneqq \mathcal{J}(u(\Omega), \Omega)$ and we consider the problem:

$$\min_{\Omega \subset \mathcal{D}} J(\Omega) \quad \text{subject to} \quad C(\Omega) = \mathbf{1}. \tag{2}$$

For numerical purposes, we need a notion of derivative with respect to the shape Ω. We thus consider a diffeomorphism $T_t \colon \overline{\mathcal{D}} \to \overline{\mathcal{D}}$, $t \in [0, \tau]$ and the associated parameterized shape $\Omega_t \coloneqq T_t(\Omega)$. The transformation must be at least bi-Lipschitz, as we will use a change of variables in integrals later to compute the shape derivative. We denote its time derivative at $t = 0$ as $\theta \coloneqq \frac{\partial}{\partial t} T_t|_{t=0}$. Conversely, one can also start with a given $\theta \in \Theta^k(\mathcal{D})$, where $\Theta^k(\mathcal{D}) \coloneqq \left\{\theta \in C^k_c(\mathbb{R}^d, \mathbb{R}^d) \;\middle|\; \theta \cdot n|_{\partial\mathcal{D}\setminus\Upsilon} = 0 \text{ and } \theta|_\Upsilon = 0\right\}$ and $\Upsilon \subset \partial\mathcal{D}$ is the set of points where the normal n is not defined, i.e., the set of singular points of $\partial\mathcal{D}$. Then, one can build $t \mapsto T_t$ of class C^1 satisfying $\frac{\partial}{\partial t} T_t|_{t=0} = \theta$ using a flow, as in the *speed method* [1, 2] or using a *perturbation of identity* [37]. The choice of the method is irrelevant for the computation of the first-order shape derivative. In any case, once T_t is available and assuming $\theta \in \Theta^k(\mathcal{D})$, we can define the shape derivative of J as follows.

Definition 1 (Shape derivative). Let $J : \mathbb{P} \to \mathbb{R}$ be a shape function.

(i) The Eulerian semiderivative of J at Ω in direction $\theta \in \Theta^k(\mathcal{D})$, when the limit exists, is defined by

$$dJ(\Omega; \theta) \coloneqq \lim_{t \searrow 0} \frac{J(\Omega_t) - J(\Omega)}{t}. \tag{3}$$

(ii) J is *shape differentiable* at Ω if it has a Eulerian semiderivative at Ω for all $\theta \in \Theta^k(\mathcal{D})$ and the mapping

$$dJ(\Omega) \colon \Theta^k(\mathcal{D}) \to \mathbb{R},\ \theta \mapsto dJ(\Omega; \theta)$$

is linear and continuous, in which case $dJ(\Omega)$ is called *shape derivative* at Ω.

When both the cost functional and the PDE constraints are expressed as bulk integrals, the corresponding shape derivative can likewise be formulated as a bulk integral, and often admits the following canonical form.

Definition 2 (Tensor representation). Let $\Omega \in \mathbb{P}$ be open. A shape differentiable function $J\colon \mathbb{P} \to \mathbb{R}$ admits a tensor representation of order 1 if there exist tensors $S_0 \in L^1(\mathcal{D}, \mathbb{R}^d)$ and $S_1 \in L^1(\mathcal{D}, \mathbb{R}^{d\times d})$, such that

$$dJ(\Omega;\theta) = \int_{\mathcal{D}} S_1 : D\theta + S_0 \cdot \theta, \tag{4}$$

for all $\theta \in \Theta^k(\mathcal{D})$.

Expression (4) is called distributed, volumetric, domain, or weak expression of the shape derivative. Tensor representations of arbitrary order can be defined in a similar way, see [27]. The order of the representation is essentially the maximal order of the differential operators appearing either in the variational formulation of the PDE constraint or in the cost functional, see [38, 39] for examples of tensor representations of order two. Tensor representations can also be formulated when the cost function and PDE constraints contain boundary terms, see [27]. In this work, we focus exclusively on representations of order one and bulk integrals in order to keep the presentation concise.

Under natural regularity assumptions, the shape derivative only depends on the restriction of the normal component $\theta \cdot n$ to the interface $\partial\Omega$. This fundamental result is known as the Hadamard–Zolésio structure theorem; see [1, pp. 480-481]. Applying the divergence theorem, this structure follows immediatly from the tensor representation (4), as shown in the following proposition.

Proposition 1 (Hadamard form)**.** *Let $\Omega \in \mathbb{P}$ and assume $\partial\Omega$ is C^2. Suppose that $dJ(\Omega)$ has the tensor representation (4). If $S_1^+ \in W^{1,1}(\Omega, \mathbb{R}^{d\times d})$ and $S_1^- \in W^{1,1}(\mathcal{D} \setminus \overline{\Omega}, \mathbb{R}^{d\times d})$, then we obtain the so-called Hadamard form or boundary expression of the shape derivative:*

$$dJ(\Omega;\theta) = \int_{\partial\Omega} G\,\theta \cdot n, \tag{5}$$

where $G := [(S_1^+ - S_1^-)n] \cdot n$ is the trace on $\partial\Omega$ of $[(S_1^+ - S_1^-)n] \cdot n$.

See [26, Proposition 1] and [27, Proposition 4.3] for proofs of Proposition 1 in more general settings. The Hadamard formula (5) is the basis of most shape derivative-based numerical methods [13, 9, 11]. It provides a natural way to express the shape derivative and derive a descent direction for optimization algorithms via the shape gradient G, and it integrates well with the original level-set method formulation [5] based on the Hamilton–Jacobi equation. However, this approach has certain drawbacks, such as stronger regularity requirements on S_1 in Proposition 1 compared to Definition 2. By contrast, the distributed shape derivative provides a natural framework for domain extension, and several recent studies have shown that it achieves higher accuracy [28, 29]. In this work, we pursue the distributed expression-based approach described in [17] using (4). A Lagrangian approach is employed to compute the tensor representation (4).

Throughout the paper, we will use the following notation for the shape derivatives of the cost and constraint functionals:

$$dJ(\Omega;\theta) = \int_{\mathcal{D}} S_0^J \cdot \theta + S_1^J : D\theta, \tag{6}$$

$$dC(\Omega;\theta) = \int_{\mathcal{D}} S_0^C \cdot \theta + S_1^C : D\theta. \tag{7}$$

3. Shape derivative for a model problem

In order to illustrate a concrete example of the calculation of a tensor representation (4), we consider a standard inverse problem in linear elasticity, where the objective is to reconstruct the shape of an inclusion with different elastic material parameters, from boundary and/or domain measurements. The uniqueness and stability of identifying a rigid inclusion in an elastic body from a boundary measurement have been investigated in [35], while the case of cavities was addressed in [40]. Shape sensitivity analysis for a related problem was studied in [36], but with a different cost functional and using strong, rather than weak, formulations of the PDEs. The emphasis in [36] is on the Hadamard form (5) of the shape derivative, rather than on the distributed formulation (4). Further examples of tensor representation of the shape derivative for other PDEs are provided in Section 7.

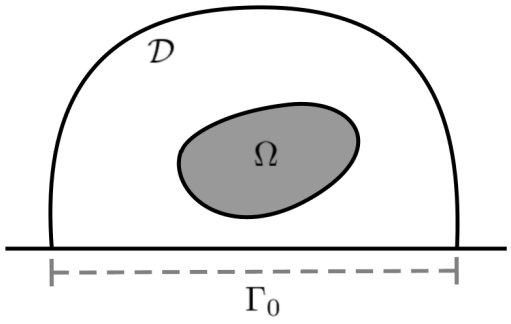


Figure 1: Geometric configuration for the inverse problem in elasticity.

Inverse problem. Let Γ_0, Γ_1 be subsets of $\Gamma \coloneqq \partial\mathcal{D}$ such that $\Gamma = \Gamma_0 \cup \Gamma_1$. Let $H_{\Gamma_0}(\mathcal{D})^d$ be the space of functions $H_{\Gamma_0}(\mathcal{D})^d \coloneqq \left\{u \in H(\mathcal{D})^d \mid u|_{\Gamma_0} = \mathbf{0}\right\}$. The strain and stress tensors are given by, respectively,

$$\epsilon(w) \coloneqq \frac{Dw + Dw^\top}{2}, \quad \sigma(w) \coloneqq \lambda \operatorname{tr}(\epsilon(w))I + 2\mu\,\epsilon(w),$$

where μ and λ are the Lamé parameters. Let $f \in H^{1/2}(\Gamma_1)^d$ and $g \in H^{1/2}(\Gamma_1)^d$ be vector-valued functions defined on Γ_1. Let α and β be positive weights.

We follow a Kohn–Vogelius type approach [41], considering the shape optimization problem:

$$\min_{\Omega\subset\mathcal{D}} J(\Omega) \coloneqq \frac{\alpha}{2}\int_{\mathcal{D}} |u-y|^2 + \frac{\beta}{2}\int_{\Gamma_1} |u-y|^2 \tag{8}$$

where u and y are the solutions to the transmission problems

$$\begin{aligned}
-\operatorname{div} A_\Omega\sigma(u) &= 0 && \text{in } \Omega \text{ and } \mathcal{D}\setminus\overline{\Omega}\\
u &= 0 && \text{on } \Gamma_0\\
A_\Omega\sigma(u)n &= f && \text{on } \Gamma_1\\
(A_\Omega\sigma(u)n)^+ &= (A_\Omega\sigma(u)n)^- && \text{on } \partial\Omega\\
u^+ &= u^- && \text{on } \partial\Omega
\end{aligned} \tag{9}$$

and

$$\begin{aligned}
-\operatorname{div} A_\Omega\sigma(y) &= 0 && \text{in } \Omega \text{ and } \mathcal{D}\setminus\overline{\Omega}\\
y &= 0 && \text{on } \Gamma_0\\
y &= g && \text{on } \Gamma_1\\
(A_\Omega\sigma(y)n)^+ &= (A_\Omega\sigma(y)n)^- && \text{on } \partial\Omega\\
y^+ &= y^- && \text{on } \partial\Omega
\end{aligned} \tag{10}$$

respectively, and $A_\Omega\colon \overline{\mathcal{D}} \to \mathbb{R}$ is given by

$$A_\Omega = \kappa\chi_\Omega + \chi_{\overline{\mathcal{D}}\setminus\Omega} \quad \text{with } \kappa > 0. \tag{11}$$

See Figure 1 for an illustration of the geometry. In (8), observe that u and y depend on Ω due to the presence of A_Ω in (9),(10). We present the model for a single measurement for simplicity. For several measurements, one can simply sum over the measurements in (8).

The state and adjoint problems. Now we write the weak formulations of (9) and (10), and of their corresponding adjoint problems. The weak formulation of (9) reads: Find $u \in H^1_{\Gamma_0}(\mathcal{D})^d$ such that

$$\int_{\mathcal{D}} A_\Omega\sigma(u) \colon \epsilon(w) = \int_{\Gamma_1} f\cdot w \quad \forall\, w \in H^1_{\Gamma_0}(\mathcal{D})^d. \tag{12}$$

It is convenient to lift the non-homogeneous Dirichlet condition $y = g$ on Γ_1 in (10). To this end, we assume that there exists an extension $g \in H^1(\mathcal{D})^d$, using the same notation for simplicity. This allows us to

rewrite (10) using the equivalent weak formulation: Find $v \in H^1_0(\mathcal{D})^d$ such that

$$\int_{\mathcal{D}} A_\Omega \sigma(v+g) : \epsilon(w) = 0 \quad \forall\, w \in H^1_0(\mathcal{D})^d, \tag{13}$$

and $v + g = y$ solves (10). Considering the cost functional (8) and the weak formulations, we write the Lagrangian functional as follows:

$$\mathcal{L}(\Omega, \varphi, \psi, \rho, \varrho) = \mathcal{L}_{\text{cost}} + \mathcal{L}_{\text{state}[f]} + \mathcal{L}_{\text{state}[g]}, \tag{14}$$

with

$$\begin{aligned}
\mathcal{L}_{\text{cost}} &= \frac{\alpha}{2} \int_{\mathcal{D}} |\varphi - \psi - g|^2 + \frac{\beta}{2} \int_{\Gamma_1} |\varphi - g|^2, \\
\mathcal{L}_{\text{state}[f]} &= \int_{\mathcal{D}} A_\Omega \sigma(\varphi) : \epsilon(\rho) - \int_{\Gamma_1} f \cdot \rho, \\
\mathcal{L}_{\text{state}[g]} &= \int_{\mathcal{D}} A_\Omega \sigma(\psi + g) : \epsilon(\varrho),
\end{aligned}$$

where $\Omega \subset \mathcal{D}$ and $\varphi, \psi, \rho, \varrho$ are functions in $H^1_{\Gamma_0}(\mathcal{D})^d$. By differentiating $\mathcal{L}$ with respect to φ and ψ, we obtain the adjoint equations: Find $p \in H^1_{\Gamma_0}(\mathcal{D})^d$ and $q \in H^1_0(\mathcal{D})^d$ such that

$$\partial_\varphi \mathcal{L}(u, v, p, q)(r) = 0 \quad \forall\, r \in H^1_{\Gamma_0}(\mathcal{D})^d, \tag{15}$$

$$\partial_\psi \mathcal{L}(u, v, p, q)(r) = 0 \quad \forall\, r \in H^1_0(\mathcal{D})^d. \tag{16}$$

From (15) we obtain the first adjoint problem: Find $p \in H^1_{\Gamma_0}(\mathcal{D})^d$ such that

$$\int_{\mathcal{D}} A_\Omega \sigma(p) : \epsilon(r) = -\alpha \int_{\mathcal{D}} (u - v - g) \cdot r - \beta \int_{\Gamma_1} (u - g) \cdot r \quad \forall\, r \in H^1_{\Gamma_0}(\mathcal{D})^d.$$

From (16) we get the second adjoint problem: Find $q \in H^1_0(\mathcal{D})^d$ such that

$$\int_{\mathcal{D}} A_\Omega \sigma(q) : \epsilon(r) = \alpha \int_{\mathcal{D}} (u - v - g) \cdot r \quad \forall\, r \in H^1_0(\mathcal{D})^d.$$

Observe that both the state and the adjoint problems are linear.

Shape derivative components. Let $T_t \colon \overline{\mathcal{D}} \to \overline{\mathcal{D}}$, $t \in [0, \tau]$ be a diffeomorphism as described in Section 2 and $\Omega_t \coloneqq T_t(\Omega)$. Let id denote the identity mapping. We assume in addition that $T_t|_{\Gamma_1} = \text{id}$ on Γ_1, where the measurements are taken. Note that the Lagrangian $\mathcal{L}$ in (14) depends on Ω_t through A_{Ω_t}. The standard procedure to compute the shape derivative is to first perform a change of variables $x \mapsto T_t(x)$ in the integrals of $\mathcal{L}$, in order to transform A_{Ω_t} into A_Ω. As this change of variables induces a composition of the variables by T_t, it is natural to reparameterize $\mathcal{L}$ by pre-composing the functions with T_t^{-1}; see [38] for a more detailed explanation. To this end, we introduce the so-called *shape-Lagrangian* as

$$\mathcal{G}(t, \varphi, \psi, \rho, \varrho) \coloneqq \mathcal{L}\left(\Omega_t, \varphi \circ T_t^{-1}, \psi \circ T_t^{-1}, \rho \circ T_t^{-1}, \varrho \circ T_t^{-1}\right)$$

for all $t \in [0, t_1]$, φ, ψ in $H^1_{\Gamma_0}(\mathcal{D})^d$, and ρ, ϱ in $H^1_0(\mathcal{D})^d$. For $t \in [0, t_1]$ and $w \in H^1(\mathcal{D})^d$, let

$$E(t, w) \coloneqq \frac{Dw\, DT_t^{-1} + (Dw\, DT_t^{-1})^\top}{2}.$$

It holds that $E(t, w) = \epsilon\left(w \circ T_t^{-1}\right) \circ T_t$. In particular, $E(0, w) = \epsilon(w)$. On the other hand, recall that $\sigma = \mathrm{C}\epsilon$, where C is the 4-index stiffness tensor associated to λ and μ. In terms of Kronecker symbols, it reads

$$\mathrm{C}_{ijkl} = \lambda \delta_{ij} \delta_{kl} + \mu(\delta_{ik}\delta_{jl} + \delta_{il}\delta_{jk}).$$

Applying the change of variable $x \mapsto T_t(x)$ in $\mathcal{G}(t, \cdot)$ and using the fact that $T_t|_{\Gamma_1} = \text{id}$ on Γ_1, we obtain

$$\mathcal{G}(t, \varphi, \psi, \rho, \varrho) = \mathcal{G}_{\text{cost}} + \mathcal{G}_{\text{state}[f]} + \mathcal{G}_{\text{state}[g]} \tag{17}$$

with

$$\mathcal{G}_{\text{cost}} = \frac{\alpha}{2}\int_{\mathcal{D}} |\varphi - \psi - g\circ T_t|^2 \xi(t) + \frac{\beta}{2}\int_{\Gamma_1} |\varphi - g|^2,$$
$$\mathcal{G}_{\text{state}[f]} = \int_{\mathcal{D}} A_\Omega \mathrm{C}E(t,\varphi) : E(t,\rho)\xi(t) - \int_{\Gamma_1} f\cdot\rho,$$
$$\mathcal{G}_{\text{state}[g]} = \int_{\mathcal{D}} A_\Omega \mathrm{C}E(t,\psi + g\circ T_t) : E(t,\varrho)\xi(t),$$

where $\xi(t) := \det DT_t$. Then, one can show that the shape derivative is given as the partial derivative of the shape-Lagrangian $\mathcal{G}$ with respect to t, using for instance the results of [1, Chapter 10, Section 5.4] or the averaged adjoint method [27]. This yields

$$dJ(\Omega;\theta) = \partial_t \mathcal{G}(0,u,v,p,q). \tag{18}$$

Using the formulas $\frac{\partial}{\partial t} DT_t^{-1}|_{t=0} = -D\theta$, $\xi'(0) = \operatorname{div}(\theta) = I : D\theta$,

$$\partial_t E(0,w) = -\frac{Dw\, D\theta + (Dw\, D\theta)^\top}{2},$$

the property $\mathrm{C}M : N = M : \mathrm{C}N$ for all , $M, N \in \mathbb{R}^{d\times d}$, and rearranging the various terms using tensor calculus [38, Lemma 2.2], we obtain

$$dJ(\Omega;\theta) = \int_{\mathcal{D}} S_0^J \cdot \theta + S_1^J : D\theta, \tag{19}$$

with S_0^J and S_1^J given by

$$S_0^J := -\alpha Dg^\top (u - v - g) + A_\Omega \sum_{i,j=1}^{d} \sigma(q)_{i,j} \nabla(\epsilon(g)_{i,j}),$$
$$S_1^J := \frac{\alpha}{2}|u - v - g|^2 I + \left(A_\Omega \sigma(u) : \epsilon(p) + A_\Omega \sigma(v+g) : \epsilon(q)\right) I$$
$$- A_\Omega \left(Du^\top \sigma(p) + Dp^\top \sigma(u) + Dv^\top \sigma(q) + Dq^\top \sigma(v+g) \right).$$

Hadamard form. Even though our numerical algorithm is based on the distributed expression (19), it is instructive to compute the Hadamard form of the shape derivative, following Proposition 1. Using (5) and the fact that the jump of $|u - v - g|^2$ on $\partial\Omega$ vanishes, we get

$$dJ(\Omega;\theta) = \int_{\partial\Omega} G\,\theta\cdot n, \tag{20}$$

with $G := \mathfrak{G}^+ - \mathfrak{G}^-$ and

$$\mathfrak{G} := A_\Omega \left\{\sigma(u) : \epsilon(p) + \sigma(v+g) : \epsilon(q)\right\} - A_\Omega \left(Du^\top \sigma(p) + Dp^\top \sigma(u) + Dv^\top \sigma(q) + Dq^\top \sigma(v+g) \right) n\cdot n \quad \text{on } \partial\Omega.$$

4. Domain approximation and error analysis

The PDEs considered in this work are posed on the hold-all domain $\mathcal{D}$, with parameters that are piecewise constant on the two phases Ω and $\mathcal{D}\setminus\Omega$. From a numerical standpoint, we employ a simple approximation Ω_h of Ω, defined as the union of mesh elements that intersect Ω. Our approach follows an optimize-then-discretize strategy, in which the shape derivative is first derived at the continuous level and then approximated numerically. As a consequence, the resulting discrete shape derivative does not coincide with the exact derivative of the discrete objective functional. Several methods have been proposed in the literature to achieve a more accurate approximation of shape derivatives on Ω, including CutFEM [42], XFEM [43], and ϕ-FEM [44]. In particular, the unfitted finite element approach introduced in [45], based on discrete level set functions, enables the computation of a discrete shape derivative that is consistent with the continuous one, thereby reconciling the discretize-then-optimize and optimize-then-discretize paradigms. These approaches, however, involve additional implementation complexity, especially in parallel computing environments, and are therefore beyond the scope of the present work. Their investigation is left for future research.

We analyze the error resulting from the approximation Ω_h of Ω using a simple two-phase problem. Let $\Omega \subset \mathcal{D}$, $f \in H^2(\mathcal{D})$, $\sigma_\Omega \colon \overline{\mathcal{D}} \to \mathbb{R}$ with $\sigma_\Omega = \sigma_0 \chi_\Omega + \sigma_1 \chi_{\overline{\mathcal{D}}\setminus\Omega}$. We assume for simplicity that Ω is sufficiently smooth and is at a positive distance of $\partial\mathcal{D}$. Consider the problem of finding $u \in H_0^1(\mathcal{D})$ such that

$$\int_{\mathcal{D}} \sigma_\Omega \nabla u \cdot \nabla v = \int_{\mathcal{D}} fv \quad \forall\, v \in H_0^1(\mathcal{D}). \tag{21}$$

The cost function is given by

$$J(\Omega) = \frac{1}{2}\int_{\mathcal{D}} \sigma_\Omega |\nabla u|^2.$$

Let $\theta \in \Theta^k(\mathcal{D})$ be a given vector field. It can be shown, see for instance [26] for a similar calculation, that the shape derivative is

$$dJ(\Omega;\theta) = \int_{\mathcal{D}} S_1 : D\theta + S_0 \cdot \theta$$

with

$$\begin{aligned} S_1 &= S_1(\Omega,u) = \left(-\frac{1}{2}\sigma_\Omega |\nabla u|^2 + fu\right) I + \sigma_\Omega \nabla u \otimes \nabla u, \\ S_0 &= S_0(\Omega,u) = u\nabla f. \end{aligned}$$

Let Ω_h be a polygonal approximation of Ω for $h \in [0,h_0]$, with $h_0 > 0$ sufficiently small. Denote by U_h the finite element solution on Ω_h obtained with P_1 elements, and by u_h the exact solution of (21) with σ_{Ω_h} in place of σ_Ω; in particular, $u_0 = u$. In a similar way, the shape derivative of J at Ω_h is given by

$$dJ(\Omega_h;\theta) = \int_{\mathcal{D}} S_1(\Omega_h,u_h) : D\theta + S_0(\Omega_h,u_h)\cdot\theta$$

and we intoduce the FE approximation of the shape derivative:

$$dJ_h(\Omega_h;\theta) := \int_{\mathcal{D}} S_1(\Omega_h,U_h) : D\theta + S_0(\Omega_h,U_h)\cdot\theta.$$

Now we assume that there exists $T^\xi \in C^1([0,h_0],\Theta^k(\mathcal{D}))$ with $\Omega_h = T^\xi(h,\Omega)$, $T_0 = \mathrm{id}$, $\frac{\partial}{\partial h} T_h^\xi|_{h=0} = \xi \in \Theta^k(\mathcal{D})$, and such that $T^\xi(h,\cdot) : \overline{\mathcal{D}} \to \overline{\mathcal{D}}$ is a bi-Lipschitz mapping for all $h \in [0,h_0]$. Note that this implies $\Omega_h \to \Omega$ in an appropriate sense. We write $T_h^\xi := T^\xi(h,\cdot)$ for simplicity. For an explicit construction of such mappings T^ξ, we refer to [46, Section 7]. For the error analysis we need the following definition.

Definition 3 (Second order shape derivative). Let $J : \mathbb{P} \to \mathbb{R}$ be a shape function, and assume that for all $t \in [0,t_0]$, $dJ(T_t^\xi(\Omega);\theta)$ exists at $T_t^\xi(\Omega)$ in direction $\theta \in \Theta^k(\mathcal{D})$.

(i) The second order Eulerian semiderivative of J at Ω in direction $\theta \in \Theta^k(\mathcal{D})$ is defined by, when the limit exists,

$$d^2J(\Omega;\theta,\xi) := \lim_{t\searrow 0} \frac{dJ(T_t^\xi(\Omega);\theta) - dJ(\Omega;\theta)}{t}. \tag{22}$$

(i) J is said to be *twice shape differentiable* at Ω if it has a second order Eulerian semiderivative at Ω for all $\theta,\xi \in \Theta^k(\mathcal{D})$ and the mapping

$$d^2J(\Omega) : [\Theta^k(\mathcal{D})]^2 \to \mathbb{R},\ (\theta,\xi) \mapsto d^2J(\Omega;\theta,\xi)$$

is bilinear and continuous, in which case $d^2J(\Omega;\theta,\xi)$ is called the *second order shape derivative* at Ω.

Let $\Omega_t = T_t^\theta(\Omega)$ and u_t be the solution of (21) with σ_{Ω_t} instead of σ_Ω. Introducing $u^t := u_t \circ T_t^\theta$, the material derivative of u is defined as $\dot{u}(\theta) = \frac{d}{dt}u^t|_{t=0}$ and it can be shown that it solves the following PDE:

$$\int_{\mathcal{D}} \sigma_\Omega \nabla \dot{u}(\theta)\cdot\nabla v = -\int_{\mathcal{D}} \sigma_\Omega \mathcal{A}'(\theta)\nabla u\cdot\nabla v + \int_{\mathcal{D}} \mathrm{div}(f\theta)v \quad \forall\, v \in H_0^1(\mathcal{D}) \tag{23}$$

where $\mathcal{A}'(\theta) := -D\theta - D\theta^\top + (\mathrm{div}\,\theta)I$; see [26, Section 6]. The proof of the following result can be adapted in a straightforward way from [26, Proposition 7].

Proposition 2. *Suppose $\Omega \in \mathbb{P}$, $f \in H^2(\mathcal{D})$, $\theta \in \Theta^2(\mathcal{D})$ and $\xi \in \Theta^1(\mathcal{D})$. The second order shape derivative in distributed form is given by*

$$d^2J(\Omega;\theta,\xi) = \int_{\mathcal{D}} S_1^2(\theta) : D\xi + S_0^2(\theta)\cdot\xi, \tag{24}$$

where

$$S_1^2(\theta) = [-\sigma_\Omega \nabla\dot{u}(\theta)\cdot\nabla u + \dot{u}(\theta)f + S_1 : D\theta + S_0\cdot\theta]I + 2\sigma_\Omega \nabla\dot{u}(\theta)\odot\nabla u + \sigma_\Omega(\nabla u\otimes\nabla u)\mathcal{A}'(\theta), \tag{25}$$

$$S_0^2(\theta) = (\dot{u}(\theta) + u\,\mathrm{div}\,\theta)\nabla f + uD^2f\theta + D^2\theta S_1 + D\theta^\top S_0, \tag{26}$$

with $S_1^2(\theta) \in L^1(\Omega,\mathbb{R}^{d\times d})$ and $S_0^2(\theta) \in L^1(\Omega,\mathbb{R}^d)$ and $D^2\theta S_1 \in \mathbb{R}^d$ is the vector with entries $[D^2\theta S_1]_k := \sum_{i,j=1}^d \partial^2_{jk}\theta_i S_{1,ij}$.

We can now prove the main result of this section.

Proposition 3. *Let $\theta \in W^{2,\infty}(\mathcal{D},\mathbb{R}^d)$ and suppose $\Omega \subset \mathcal{D}$ is Lipschitz. Let $T^\xi \in C^1([0,h_0],\Theta^k(\mathcal{D}))$ be such that $T_0 = \mathrm{id}$, $\frac{\partial}{\partial h}T_h^\xi|_{h=0} = \xi \in \Theta^k(\mathcal{D})$, and $T^\xi(h,\cdot) : \overline{\mathcal{D}} \to \overline{\mathcal{D}}$ is a bi-Lipschitz mapping for all $h \in [0,h_0]$. Then, with $\Omega_h := T^\xi(h,\Omega)$ we have*

$$|dJ(\Omega;\theta) - dJ_h(\Omega_h;\theta)| \le Ch\|\theta\|_{W^{2,\infty}(\mathcal{D})}.$$

Proof. Throughout this proof, the notation C denotes a generic constant. We start with the splitting

$$|dJ(\Omega;\theta) - dJ_h(\Omega_h;\theta)| \le |dJ(\Omega;\theta) - dJ(\Omega_h;\theta)| + |dJ(\Omega_h;\theta) - dJ_h(\Omega_h;\theta)|. \tag{27}$$

In view of Definition 3, we have

$$dJ(\Omega;\theta) - dJ(\Omega_h;\theta) = hd^2J(\Omega;\theta,\xi) + O(h^2)$$

recalling that $\xi = \frac{d}{dh}T_h^\xi|_{h=0}$ and $T_h^\xi := T^\xi(h,\cdot)$. In view of (21) and (23) one can prove the estimates $\|u\|_{H^1(\mathcal{D})} \le C$ and $\|\dot{u}(\theta)\|_{H^1(\mathcal{D})} \le C\|\theta\|_{W^{1,\infty}(\mathcal{D})}$. Thus, using (25),(26) and the regularity assumptions on f,θ,ξ we get

$$|d^2J(\Omega;\theta,\xi)| \le C\|\theta\|_{W^{2,\infty}(\mathcal{D})}\|\xi\|_{W^{1,\infty}(\mathcal{D})}.$$

This yields, for h sufficiently small,

$$|dJ(\Omega;\theta) - dJ(\Omega_h;\theta)| \le Ch\|\theta\|_{W^{2,\infty}(\mathcal{D})}\|\xi\|_{W^{1,\infty}(\mathcal{D})}.$$

For the other term in (27) we get

$$|dJ(\Omega_h;\theta) - dJ_h(\Omega_h;\theta)| = \left|\int_{\mathcal{D}} (S_1(\Omega_h,u_h) - S_1(\Omega_h,U_h)) : D\theta + (S_0(\Omega_h,u_h) - S_0(\Omega_h,U_h))\cdot\theta\right|.$$

We have

$$\begin{aligned} S_1(\Omega_h,u_h) - S_1(\Omega_h,U_h) = &\left(-\frac{1}{2}\sigma_{\Omega_h}(|\nabla u_h|^2 - |\nabla U_h|^2) + f(u_h - U_h)\right) I \\ &+ \sigma_{\Omega_h}(\nabla u_h\otimes\nabla u_h - \nabla U_h\otimes\nabla U_h). \end{aligned}$$

For the first term, using the uniform boundedness of σ_{Ω_h} in $L^\infty(\mathcal{D})$ and the uniform boundedness of u_h, U_h in $H^1(\mathcal{D})$, we have the estimate

$$\begin{aligned} \left|\int_{\mathcal{D}} -\frac{1}{2}\sigma_{\Omega_h}(|\nabla u_h|^2 - |\nabla U_h|^2)D\theta : I\right| &= \left|\int_{\mathcal{D}} -\frac{1}{2}\sigma_{\Omega_h}(\nabla u_h - \nabla U_h)\cdot(\nabla u_h + \nabla U_h)D\theta : I\right| \\ &\le C\|u_h - U_h\|_{H^1(\mathcal{D})}\|\theta\|_{W^{1,\infty}(\mathcal{D})}. \end{aligned}$$

The other terms can be treated in a similar way and we obtain

$$|dJ(\Omega_h;\theta) - dJ_h(\Omega_h;\theta)| \le C\|u_h - U_h\|_{H^1(\mathcal{D})}\|\theta\|_{W^{1,\infty}(\mathcal{D})}.$$

Standard P1 finite element estimates yields

$$\|u_h - U_h\|_{H^1(\mathcal{D})} \leq Ch\|u_h\|_{H^1(\mathcal{D})}.$$

Thus, we have obtained

$$|dJ(\Omega_h;\theta) - dJ_h(\Omega_h;\theta)| \leq Ch\|\theta\|_{W^{1,\infty}(\mathcal{D})}.$$

Gathering these results we have proved

$$|dJ(\Omega;\theta) - dJ_h(\Omega_h;\theta)| \leq Ch\|\theta\|_{W^{2,\infty}(\mathcal{D})}\|\xi\|_{W^{1,\infty}(\mathcal{D})} + Ch\|\theta\|_{W^{1,\infty}(\mathcal{D})} \leq Ch\|\theta\|_{W^{2,\infty}(\mathcal{D})},$$

which yields the result. □

The proof methodology of Proposition 3 is general and can be adapted straightforwardly to other setting, in particular to elasticity with ersatz material. Problems with only one phase Ω can also be treated in a similar way.

5. Main algorithm

In this section we describe the main steps of the algorithm, which can be summarized as follows. We employ the level set method, introduced by Osher and Sethian [5], to update Ω. In this method, the shape Ω is represented as the subzero level set of a function $\phi\colon \overline{\mathcal{D}} \to \mathbb{R}$, namely

$$\Omega \coloneqq \{x \in \mathcal{D} \mid \phi(x) < 0\}. \tag{28}$$

For a given ϕ, we start by solving the state and adjoint equations, which are then used to construct the shape derivative components of (4), first computed analytically. Next, a descent direction $\theta\colon \overline{\mathcal{D}} \to \mathbb{R}^d$ is computed by solving the so-called *velocity equation*: Find $\theta \in \mathbb{H}$ such that

$$B(\theta,\xi) = -dJ(\Omega;\xi) \quad \forall\, \xi \in \mathbb{H}, \tag{29}$$

where $dJ(\Omega;\xi)$ is the distributed expression (4) and $B : \mathbb{H}\times\mathbb{H} \to \mathbb{R}$ is a user-defined positive definite bilinear form, with $\mathbb{H}$ an appropriate function space. The solution is indeed a descent direction, as $dJ(\Omega;\theta) = -B(\theta,\theta) < 0$. The implementation is described in Subsection 6.5. The computation of a descent direction via the solution of (29) is standard in level set methods, also when using a boundary expression (Hadamard form) of the shape derivative instead of a distributed expression. In this context, the H^1-metric employed in this work is a commonly used choice. The bilinear form typically has the form:

$$B(\theta,\xi) = \alpha_1 \int_{\mathcal{D}} \theta\cdot\xi + \alpha_2 \int_{\mathcal{D}} D\theta : D\xi + \alpha_3 \int_{\partial\mathcal{D}} (\theta\cdot n)(\xi\cdot n), \tag{30}$$

where $\alpha_1, \alpha_2 > 0$ and $\alpha_3 \geq 0$ are user-defined parameters. The term associated with α_2 acts as a regularization, promoting smoothness of the descent direction, while the boundary term weighted by α_3 penalizes normal displacements on $\partial\mathcal{D}$, but allows tangential motion along the boundary. Usually, the influence of the specific choice of B on the convergence is not studied, but there are a few references that address this question. Early work on the use of different inner products for computing shape gradients from boundary expressions can be found in [47]; see also [25]. A notable more recent contribution is [48], where reproducing kernel Hilbert spaces are employed to define gradient-based algorithms with varying metrics, using distributed shape derivatives. In that work, the influence of the metric choice on convergence is also investigated numerically. Section 7.5 also presents a numerical study of how the algorithm's convergence behavior depends on the choice of B.

Further, θ is used to update the level set function by solving the following transport-like equation on a short time interval:

$$\begin{aligned} \partial_t\phi + \theta\cdot\nabla\phi = h^2\Delta\phi \quad & x \in \mathcal{D},\, t > 0, \\ \partial_n\phi = 0 \quad & x \in \partial\mathcal{D},\, t > 0, \\ \phi(0,x) = \phi^i(x) \quad & x \in \overline{\mathcal{D}}, \end{aligned} \tag{31}$$

where ϕ^i is the current iteration and h is the mesh diameter. The implementation is described in Subsection 6.6.

The key steps of the level set method are summarized in Algorithm 1. The numerical implementation also involves a reinitialization, a Lagrangian approach for handling constraints, and a stopping criterion, which will be explained in the following subsections.

Algorithm 1 Level set algorithm

Initialization: Choose an initial $\phi^0 : \overline{\mathcal{D}} \to \mathbb{R}$.
for $i = 0, 1, 2, \ldots$ **do**
    Solve **state** problems with Ω^i. Let U be the solutions.
    Solve **adjoint** problems with Ω^i and U. Let P be the solutions.
    Use Ω^i and U to compute **cost** value $J(\Omega^i)$.
    Use Ω^i and U to compute **constraint** error.
    Check stopping criterion based on cost value and constraint error.
    Use Ω^i, U, P to build derivative components: S_0^J, S_1^J (cost)and S_0^C, S_1^C (constraint).
    Solve **velocity** equation (29) with S_0^J, S_1^J, S_0^C, and S_1^C. Let $\theta \colon \overline{\mathcal{D}} \to \mathbb{R}^d$ be the solution.
    Solve **transport** equation (31) with θ. Let $\phi^{i+1} \colon \overline{\mathcal{D}} \to \mathbb{R}$ be the solution.
    Check stopping criterion.
end for

6. Numerical implementation

The Python module `formopt` was developed to apply the level set method to shape optimization problems. It implements the level set algorithm (see Algorithm 1) using the finite element method to solve the weak formulations of all problems involved. The shape optimization problem is defined as a model that includes all relevant equations and the distributed shape derivatives, while sub-problems are addressed using parallel computing methodologies.

This section covers four aspects: the main classes and functions provided by `formopt`, the construction of the model problem, the three parallelism paradigms that were implemented, and the numerical methods used to solve the sub-problems.

6.1. Toolbox structure

The `formopt` toolbox is organized around a modular structure designed to reflect the main components of the optimization algorithm. At its core lies an abstract `Model` class, which defines the problem-specific ingredients of the optimization procedure. In particular, user-defined subclasses must provide the weak formulations of the state and adjoint equations, the cost functional and constraints, the distributed shape derivative, and the bilinear form used to compute the descent direction. This design allows a clear separation between problem definition and algorithmic implementation. We have implemented several models in `models.py`, see the description in Table 1.

The computation of the descent direction is handled by a dedicated `Velocity` class, which assembles and solves the velocity equation (29). Since the bilinear form B remains fixed throughout the iterations, the corresponding system matrix is assembled once, while only the right-hand side is updated at each iteration.

The evolution of the domain is performed using a `Level` class, which solves the transport equation (31) governing the level set function. The associated discrete problem is assembled efficiently by reusing precompiled components whenever possible. Additional modules handle auxiliary tasks such as initialization of the level set function, reinitialization procedures, adaptive time stepping, and constraint handling via an augmented Lagrangian approach.

The main functions in the `formopt` module implement Algorithm 1 using different parallelism modes: `runDP` for data parallelism, `runTP` for task parallelism and `runMP` for mixed parallelism. These strategies are handled transparently at the algorithmic level, allowing the user to define the model independently of the chosen parallelism paradigm.

A detailed description of the software architecture, including class interfaces, function signatures, and implementation details, is provided in Appendix A.1.

6.2. Model construction

A problem is defined in the `formopt` framework by specifying a set of core ingredients through a model class. In particular, the user must provide the weak formulation of the state equations, the corresponding adjoint equations, the cost functional $J(\Omega)$, possible constraints $C(\Omega)$, the distributed shape derivative $dJ(\Omega; \theta)$, expressed through its tensor components, and the bilinear form B defining the metric used to compute the descent direction. The main function arguments in this model class are the level set function ϕ, the state variables, and the adjoint variables. The state and adjoint problems are formulated in weak form,

Model	Description
`Compliance`	compliance minimization with one load
`CompliancePlus`	similar to `Compliance`, but with multiple loads
`InverseElasticity`	inverse problem in linear elasticity
`Heat`	heat conduction problem
`HeatPlus`	same as `Heat`, but with multiple sinks and sources
`Logistic`	resource distribution for a population governed by a nonlinear logistic equation

Table 1: Models implemented in `models.py`.

while the shape derivative is represented in its distributed form using S_0, S_1 introduced in Section 2. These components are implemented using the Unified Form Language (UFL), which allows a direct transcription of variational formulations into code. A detailed description of the corresponding class structure, including method signatures and implementation details, is provided in Appendix A.2.

6.3. Parallelization

The main principle of parallel computing is to break a large problem into smaller tasks that can be solved independently and simultaneously using multiple processors, typically CPU cores. As the processing power of a single CPU has nearly reached its physical limits, and as data sizes increase and simulations become more detailed, computing in parallel has become essential to overcome these limitations. The main challenges in parallel computing are how to divide the problem effectively and how to manage communication and coordination between processors. These must be handled efficiently to ensure that parallelization leads to a significant reduction in computation time.

Our module supports three parallelism paradigms: *data parallelism* (`DP`), *task parallelism* (`TP`), and a combination of the two, known as *mixed parallelism* (`MP`). *Data parallelism* refers to solving the problem on a mesh that is distributed across multiple processes. One uses domain decomposition methods to partition the domain and solve the PDE locally in each part [49]. All the weak formulations are solved on this distributed mesh. In the *task parallelism* paradigm, each state equation is solved in a separate process, and the same applies to each adjoint equation. In this case, each process must have its own copy of the mesh. The velocity field and the level set function are computed in the first process (identified as `rank = 0`). The *mixed parallelism* paradigm combines these two modes. The processes are partitioned into groups of equal size, with each group assigned to a single state equation. Within a group, the corresponding state equation is solved on a mesh distributed across multiple subprocesses. Subprocesses belonging to the same group share the same marker `color`. The adjoint equations are treated analogously. The velocity field and the level set function, however, are computed within the first group of subprocesses (identified by `color = 0`).

Implementation details, including communicator definitions and domain construction, are provided in Appendix A.3.

6.4. Lagrangian method

To handle the constraints $C(\Omega) = \mathbf{1}$, we implement the Lagrangian-based first-order method developed in [50]. We begin by formulating the Proximal-Perturbed Lagrangian

$$L(\Omega, z, \lambda, \mu) := J(\Omega) + \langle \lambda, C(\Omega) - \mathbf{1} \rangle + \langle \mu, z \rangle + \frac{\alpha}{2}|z|^2 + \frac{\beta}{2}|\lambda - \mu|^2, \tag{32}$$

where z is a perturbation variable, λ and μ are Lagrange multipliers, α is a penalty parameter, and β is a proximal parameter. Here, $\langle \cdot, \cdot \rangle$ and $|\cdot|$ are the standard inner product and norm of $\mathbb{R}^{N_c}$. We then follow [50, Algorithm 1]: Given the parameters $r \in (0,1)$, $\alpha \in (1, \infty)$, $\beta \in (0,1)$, $\delta^0 \in (0,1]$, and initial values (z^0, λ^0, μ^0), the iterations are defined by

$$\begin{aligned} \mu^i &= \mu^{i-1} + \delta^{i-1} \tfrac{\lambda^{i-1} - \mu^{i-1}}{|\lambda^{i-1} - \mu^{i-1}|^2 + 1}, & \lambda^i &= \mu^i + \tfrac{\alpha}{1+\alpha\beta}(C(\Omega^i) - \mathbf{1}) \\ z^i &= \tfrac{1}{\alpha}(\lambda^i - \mu^i), & \delta^i &= r\delta^{i-1} \end{aligned} \tag{33}$$

for $i = 1, 2, \ldots$, where Ω^i is given by (28). Note that, in our implementation, the update of the level set function via the transport equation (31) replaces the gradient descent step in [50, Algorithm 1], which aims to minimize L with respect to its first primal variable (here Ω).

The Lagrange multiplier λ^i is used to construct the shape derivative components S_0 and S_1 given by

$$S_0 \coloneqq S_0^J + \lambda^i S_0^C, \quad S_1 \coloneqq S_1^J + \lambda^i S_1^C; \tag{34}$$

see (6) and (7). Our module recognizes the number of constraints by calling the method `constraints` of the user-defined class. If there are no constraints (i.e., `constraints` returns an empty list), then the Lagrangian method is not applied, and we simply set $S_0 = S_0^J$ and $S_1 = S_1^J$.

In the initialization of the `PPL` class, we set the parameter values as in [50, Example 3]: $r = 0.999$, $\delta^0 = 0.5$, $\alpha = 2000$, $\beta = 0.5$, $z^0 = 0$, $\lambda^0 = 0$, and $\mu^0 = 0$. These values are used in all our tests and yield satisfactory results, although alternative values can be specified in the `__init__` method of the `PPL` class. The choice $z^0 = 0$ is natural, since we expect the constraint function to be approximately satisfied at the end of the iterations. The initial guesses for the Lagrange multipliers could be further refined by considering the ratio of the cost to the constraint at the initial iteration. The penalty parameter α is chosen large to enforce the constraint via the perturbation term, while the proximal parameter β is taken small to facilitate the identification of the multipliers during the iterations.

6.5. Velocity

In view of (29), the velocity field θ is obtained by solving the following weak formulation: Find $\theta \in \mathbb{H}$ such that

$$B(\theta, \xi) = -\int_{\mathcal{D}} S_0 \cdot \xi + S_1 : D\xi \quad \forall\, \xi \in \mathbb{H}, \tag{35}$$

where S_0 and S_1 are defined in (34), and the Hilbert space $\mathbb{H}$ is either $H^1(\mathcal{D})^d$ or $H_0^1(\mathcal{D})^d$.

The `bilinear_form` method must return the UFL expression of bilinear form B, along with a Boolean value indicating whether homogeneous Dirichlet boundary conditions should be imposed: `False` if $\mathbb{H} = H^1(\mathcal{D})^d$ and `True` if $\mathbb{H} = H_0^1(\mathcal{D})^d$. In the case $\mathbb{H} = H^1(\mathcal{D})^d$, we recommend adding to B a term of the form

$$10^4 \int_{\partial\mathcal{D}} (\theta \cdot n)(\xi \cdot n) \tag{36}$$

to penalize the normal component of θ on the boundary. This enforces the velocity field θ to be (almost) tangential along the boundary. For instance, (36) is written as `B+=1e4*dot(th,nv)*dot(xi,nv)*self.ds`, where `B` contain the other terms of the bilinear form; then `bilinear_form` must return `B,False`.

Recall that the `Velocity` class sets up and solve (35) internally. The outputs of the `derivative` and `bilinear_form` methods are sufficient to configure this part.

6.6. Transport equation

In the standard level set method [5], the level set function is updated by solving a Hamilton–Jacobi equation over a short time interval, using a descent direction $\theta \cdot n$ in the normal direction, derived from the boundary expression. In the distributed shape derivative-based level set method, one rather solves a linear transport equation, as θ is available in $\mathcal{D}$ rather than $\theta \cdot n$ on $\partial\Omega$, see [27]. Accordingly, we update the level set function by solving the diffusion–transport problem 31. Note that the diffusion term $h^2\Delta\phi$ was added to the linear transport equation (31). Indeed, during the shape optimization process, small artificial interfaces can sometimes persist between regions that are expected to merge. These thin "ghost boundaries" prevent the complete merging of nearby holes, leading to non-smooth intermediate geometries. The additional diffusion smooths the level set evolution and removes spurious residual interfaces. Similar regularization mechanisms via diffusion terms have also been employed in reaction–diffusion-based level set methods; see [51].

From a theoretical viewpoint, this diffusion term is consistent with the notion of *viscosity solution* for Hamilton-Jacobi equations; see [52] for a definition. In this framework, solutions of parabolic regularizations of the form $\partial_t\phi_\varepsilon + H(\nabla\phi_\varepsilon) = \varepsilon\Delta\phi_\varepsilon$ are known to converge, as $\varepsilon \to 0$, to the viscosity solution of the corresponding first-order equation under suitable assumptions; see [52, Theorem 3.1]. Nevertheless, in the present setting the diffusion coefficient is tied to the mesh size ($\varepsilon = h^2$), so the term should primarily be viewed as a discretization-dependent regularization. Consequently, it introduces additional smoothing compared to standard Hamilton–Jacobi schemes, which may slightly alter the interface location while improving numerical robustness. We provide here a formal justification for this regularization. Let θ be a given descent direction for the cost function $J(\Omega)$ at a given Ω, i.e., we have $dJ(\Omega;\theta) < 0$. Assume for simplicity that Ω is sufficiently smooth and lies at a positive distance from $\mathcal{D}$ and recall that n is the unit normal vector to $\partial\Omega$, pointing

outwards of Ω. Let $\hat{\phi}$ be the solution of (31) without the diffusion term $h^2\Delta\phi$, and with the initialization $\Omega = \{x \in \mathcal{D} : \hat{\phi}_0(x) < 0\}$, where $\hat{\phi}_0 := \hat{\phi}(0, \cdot)$. Denote $\widehat{\Omega}_t := \{x \in \mathcal{D} : \hat{\phi}(t, x) < 0\}$. We consider the solution on a sufficiently small time interval $[0, t_0]$, so that no topological changes occur. Without loss of generality, we can assume that $|\nabla\hat{\phi}_0| \equiv 1$ in a neighbourhood of $\partial\Omega$. It can be shown, using [53, Theorem 3], that we have the following expansion of the cost function:

$$J(\widehat{\Omega}_t) = J(\Omega) + tdJ(\Omega; \hat{\theta}) + O(t^2),$$

with $\hat{\theta} := -(\theta \cdot \nabla\hat{\phi}_0)|\nabla\hat{\phi}_0|^{-2}\nabla\hat{\phi}_0$ in $\mathcal{D}$. Since $|\nabla\hat{\phi}_0| \equiv 1$ on $\partial\Omega$, we have $\nabla\hat{\phi}_0 = n$ on $\partial\Omega$, hence $\hat{\theta}|_{\partial\Omega} := (\theta \cdot n)n$ on $\partial\Omega$. Since Ω is sufficiently smooth, this shows that $dJ(\Omega; \hat{\theta}) = dJ(\Omega; \theta) < 0$, hence $\hat{\theta}$ is also a descent direction.

Let us now consider the case where ϕ is the solution of (31) with the diffusion term $h^2\Delta\phi$. Denote this time $\Omega_t := \{x \in \mathcal{D} : \phi(t, x) < 0\}$. In a similar way, using [53, Theorem 3], one can show the expansion

$$J(\Omega_t) = J(\Omega) + tdJ(\Omega)(\hat{\theta}_\varepsilon) + O(t^2)$$

with $\varepsilon = h^2$ and $\hat{\theta}_\varepsilon := -(\theta \cdot \nabla\hat{\phi}_0 - \varepsilon\Delta\hat{\phi}_0)|\nabla\hat{\phi}_0|^{-2}\nabla\hat{\phi}_0$ in $\mathcal{D}$. Using $|\nabla\hat{\phi}_0| \equiv 1$ on $\partial\Omega$ we get

$$\hat{\theta}_\varepsilon|_{\partial\Omega} = (\theta \cdot n)n - \varepsilon(\Delta\hat{\phi}_0)n \text{ on } \partial\Omega.$$

Thus, $\hat{\theta}_\varepsilon$ is not a descent direction in general, but converges towards the descent direction $\hat{\theta} = (\theta \cdot n)n$ as $\varepsilon = h^2 \to 0$, i.e., as the mesh is refined. We can further analyze $\hat{\theta}_\varepsilon$ as follows. Let $\mathcal{H} := \frac{\mathrm{div}_\Gamma\, n}{d-1}$ be the mean curvature of $\partial\Omega$, where div_Γ denotes the tangential divergence. Using [37, Proposition 5.4.12] we have

$$\Delta\hat{\phi}_0 = \Delta_\Gamma\hat{\phi}_0 + (d-1)\mathcal{H}\partial_n\hat{\phi}_0 + D^2\hat{\phi}_0 n \cdot n \text{ on } \partial\Omega,$$

where Δ_Γ is the Laplace-Beltrami operator. Differentiating $|\nabla\hat{\phi}_0|^2 \equiv 1$ in a neighbourhood of $\partial\Omega$ we get $D^2\hat{\phi}_0\nabla\hat{\phi}_0 = 0$ on $\partial\Omega$, hence $D^2\hat{\phi}_0 n \cdot n = 0$ on $\partial\Omega$. We also have $\Delta_\Gamma\hat{\phi}_0 = 0$ and $\partial_n\hat{\phi}_0 = 1$ on $\partial\Omega$, hence $\Delta\hat{\phi}_0 = (d-1)\mathcal{H}$ on $\partial\Omega$. Thus, we get

$$\hat{\theta}_\varepsilon|_{\partial\Omega} = (\theta \cdot n)n - \varepsilon\mathcal{H}.$$

This shows that the diffusion term in (31) induces a curvature-driven regularization in the interface evolution. Since the curvature represents the shape gradient of the perimeter of $\partial\Omega$, this can be interpreted as adding an implicit perimeter penalization with weight $\varepsilon = h^2$ to the cost function. Therefore, although this approach deviates from a strict optimize-then-discretize paradigm, it can be viewed as a controlled, vanishing perturbation of the descent direction. This perturbation improves numerical robustness while only mildly affecting the resulting optimization trajectory.

Note that the diffusion term in (31) is optional and can be added or removed by setting `True` or `False`, respectively, to the parameter `smooth`, which is passed as an argument to the `run<DP|TP|MP>` functions. By default, `smooth=False`. We recommend keeping `smooth=False` for heat conduction problems where the goal is to obtain tree-like material morphologies. In contrast, for compliance minimization problems, we suggest setting `smooth=True` to suppress the formation of spurious residual interfaces.

The mesh diameter h is defined as

$$h := \begin{cases} 4\frac{|\mathcal{D}|}{N_T\sqrt{3}} & \text{if } d = 2, \\ \left(6\sqrt{2}\frac{|\mathcal{D}|}{N_T}\right)^{2/3} & \text{if } d = 3, \end{cases}$$

where $|\mathcal{D}|$ denotes the area ($d = 2$) or volume ($d = 3$) of the domain $\mathcal{D}$, and N_T denotes the number of triangles ($d = 2$) or tetrahedra ($d = 3$). This expression provides an approximation of the maximum diameter of the finite elements, assuming that the elements are identical equilateral triangles or regular tetrahedra.

We apply the Petrov–Galerkin method [54] to (31), along with the Crank-Nicolson method to the time derivative. This results in iteratively solving the following weak formulation: Find $\phi(t + \delta t, \cdot) \in H^1(\mathcal{D})$ such that

$$\int_{\mathcal{D}} \phi(t + \delta t, \cdot)\hat{\psi} + \delta t\, \mathcal{F}(\phi(t + \delta t, \cdot), \hat{\psi}) = \int_{\mathcal{D}} \phi(t, \cdot)\hat{\psi} - \delta t\, \mathcal{F}(\phi(t, \cdot), \hat{\psi}) \tag{37}$$

for all $\psi \in H^1(\mathcal{D})$, where δt is the time step,

$$\mathcal{F}(u,v) = \frac{(\theta\cdot\nabla u)v}{2} + h^2\frac{\nabla u\cdot\nabla v}{2}$$

and $\hat{\psi} = \psi + \tau\theta\cdot\nabla\psi$, with

$$\tau(x) = \frac{1}{2}\left(\frac{1}{\delta t^2} + \frac{|\theta(x)|^2}{h^2}\right)^{-1/2}. \tag{38}$$

This choice of τ is motivated by standard practice for conservation laws, where τ serves as a stabilization parameter that controls numerical dissipation along characteristic directions. We solve (37) up to a final time t_{end} and define the new level set function as $\phi^{i+1}(x) \coloneqq \phi(t_{\text{end}}, x)$.

The Neumann boundary condition in (31) serves to eliminate the boundary integral term of the weak form of the diffusion term. Moreover, no inflow boundary condition is considered in (31) since we assume that the velocity field θ is either $\theta = 0$ or $\theta\cdot n \approx 0$ on Γ.

The implementation and resolution of (37) are handled as in the `Level` class.

6.7. Other implementation aspects

Reinitialization. The goal of reinitialization in the standard level set method [5], see also [17], is to prevent the level set function from degenerating over successive iterations. This means that one strives to preserve the property $|\nabla\phi| \approx 1$, at least in the vicinity of the interface. We follow the same approach and regularly reinitialize the level set function $\phi = \phi(t_{\text{end}}, \cdot)$ (obtained from (31)) by solving the diffusive Eikonal equation with pseudo-time derivative:

$$\begin{aligned} \partial_t\varphi - h^2\Delta\varphi = S(\phi)(1-|\nabla\varphi|) \quad & x\in\mathcal{D},\ t>0,\\ \partial_n\varphi = 0 \quad & x\in\partial\mathcal{D},\ t>0,\\ \varphi(0,x) = \phi(x) \quad & x\in\overline{\mathcal{D}}, \end{aligned} \tag{39}$$

where $S(\phi)\colon \overline{\mathcal{D}} \to (-1,1)$ is given by $S(\phi) = \frac{\phi}{\sqrt{\phi^2+h^2}}$ (h = mesh diameter). The function $S(\phi)$ approximates the signed distance function associated with the set $\{x\in\overline{\mathcal{D}} \mid \phi(x) < 0\}$. We write the first equation in (39) as a Hamilton–Jacobi equation:

$$\partial_t\varphi + H(\nabla\varphi) = S(\phi) + h^2\Delta\varphi \quad x\in\mathcal{D},\ t>0, \tag{40}$$

where $H(p) \coloneqq S(\phi)|p|$ is the Hamiltonian. Note that H is homogeneous of degree 1: $H(\lambda p) = \lambda H(p)$ for all positive λ. By Euler's homogeneous function theorem, $H(p) = \nabla H(p)\cdot p$, where

$$\nabla H(p) = S(\phi)\frac{p}{|p|}. \tag{41}$$

These observations lead us to consider again the Petrov–Galerkin method developed in [54]. Discretizing the time derivative with the two-step Adams–Bashforth method, we obtain the following iterative scheme: Find $\varphi(t+\delta t,\cdot)\in H^1(\mathcal{D})$ such that

$$\int_{\mathcal{D}}\varphi(t+\delta t,\cdot)\hat{\psi} = \int_{\mathcal{D}}(\varphi(t,\cdot)+\delta t\, S(\phi))\hat{\psi} + \int_{\mathcal{D}}\delta t\,\mathcal{G}(\varphi(t,\cdot),\varphi(t-\delta t,\cdot),\hat{\psi}) \tag{42}$$

for all $\psi\in H^1(\mathcal{D})$, where δt is the time step, $\mathcal{G}(u,v,w) = \frac{H(\nabla v)-3H(\nabla u)}{2}w + h^2\frac{\nabla v-3\nabla u}{2}\cdot\nabla w$, and $\hat{\psi} = \psi + \tau\nabla H(\nabla\varphi(t,\cdot))\cdot\nabla\psi$, with

$$\tau(x) = \frac{1}{2}\left(\frac{1}{\delta t^2} + \frac{|\nabla H(\nabla\varphi(t,x))|^2}{h^2}\right)^{-1/2}. \tag{43}$$

Note that $|\nabla H(\nabla\varphi(t,\cdot))| = |S(\phi)|$ (set $p = \nabla\varphi$ in (41)), so τ is time-independent, just as in (38). The Adams–Bashforth method provides an explicit scheme to the nonlinear equation (39), with second-order accuracy in time. The diffusion term prevents the propagation of small instabilities due to $\nabla H(\nabla\varphi(t,\cdot))$.

Although φ approximates the distance function associated to $\{x \in \overline{\mathcal{D}} \mid \phi(x) < 0\}$, and therefore ideally satisfies $|\nabla\varphi| \approx 1$, this may not hold during the first iterations of (42) due to the initial condition.

In practice, we observed that computing the first iterate $\varphi(\delta t, \cdot)$ using the explicit Euler method is sufficient to start the scheme. Finally, we point out that, unlike in [54], inflow fluxes were not considered.

Recall that the `Reinit` class implements the reinitialization. Its constructor creates a precompiled solver for (42), and the method `run` performs the iterations. Two user-defined parameters are available in `run<DP|TP|MP>` to configure the reinitialization:

- `reinit_step`: Either `False` or a positive integer. If `False`, no reinitialization is performed. Otherwise the reinitialization is performed whenever `reinit_step%i=0`, where `i` denotes the current iteration. By default, `reinit_step=False`.
- `reinit_pars`: Tuple with the number of iterations and the final time for (42). By default, we set `reinit_pars=(8,1e-1)`.

Adaptive time-stepping. The final time and the number of steps/iterations in (37) are chosen as

$$t_{\text{end}} = \frac{\texttt{dfactor}}{B(\theta,\theta)} \tag{44}$$

and

$$\hat{s} = (s_{\max} - s_{\min}) \left(\frac{t_{\text{end}} - t_{\min}}{t_{\max} - t_{\min}}\right)^{1/6} + s_{\min}, \tag{45}$$

respectively, where θ is the velocity field solution to (35). The other parameters are user-defined:

- `dfactor`: Positive float to scale the inverse of $B(\theta,\theta)$. We recommend choosing `dfactor` ≤ 1. By default, `dfactor=1e-2`.
- `lv_time` $= (t_{\min}, t_{\max})$: Tuple with the minimum and maximum times allowed. By default, we set `lv_time=(1e-3,1e-1)`.
- `lv_iter` $= (s_{\min}, s_{\max})$: Tuple with the minimum and maximum number of steps allowed. By default, `lv_iter=(8,16)`.

These parameters are configured in `run<DP|TP|MP>`. Note that in (44) the final time t_{end} varies inversely with the magnitude of θ, which carries information about the shape derivative components. In (45), we apply a concave increasing function to t_{end} in order to determine the number of steps, allowing a few more steps for larger values of $B(\theta,\theta)$.

To asses the robustness of the adaptive time-stepping, randomness can be introduced by specifying a seed number N_{seed} and a noise level ϑ in the argument `random_pars` $= (N_{\text{seed}}, \vartheta)$. Then, t_{end} is scaled by a random factor drawn from the uniform distribution $\mathcal{U}(1-\vartheta, 1+\vartheta)$ and $\hat{s}$ is replaced by a sample drawn from the normal distribution $\mathcal{N}(\hat{s}, (\vartheta\hat{s})^2)$. By default, `random_pars=(1, 0.05)`.

In addition, a lower bound is imposed on $B(\theta,\theta)$ to prevent division by zero in (44). Before returning t_{end} and $\hat{s}$, we ensure that $t_{\min} \leq t_{\text{end}} \leq t_{\max}$, $s_{\min} \leq \hat{s} \leq s_{\max}$, and that $\hat{s}$ is rounded to an integer. We recall that the `AdapTime` class performs all these procedures.

Stopping criterion. Starting from iteration i of Algorithm 1 such that $i >$ `start_to_check`, we apply the following stopping criteria:

- (problem without constraints) the relative error of the cost functional J, namely

$$\left|(J(\Omega^i) - J(\Omega^j))_{j\in I(i)}\right|_\infty < \texttt{cost\_tol} \cdot |J(\Omega^i)|;$$

- (problem with constraints) the relative error of the Lagrangian functional L together with the constraint error, namely

$$\left|(L(\Omega^i, z^i, \lambda^i, \mu^i) - L(\Omega^j, z^j, \lambda^j, \mu^j))_{j\in I(i)}\right|_\infty < \texttt{lgrn\_tol} \cdot |L(\Omega^i, z^i, \lambda^i, \mu^i)|$$

  and $|C(\Omega^i) - \mathbf{1}|_\infty <$ `ctrn_tol`,

where $I(i) = \{i - \texttt{prev}, \dots, i-1\}$. The parameters `cost_tol`, `lgrn_tol`, and `ctrn_tol` are error tolerances, and `prev` is the number of previous values considered. The default parameter values are `start_to_check=30`, `cost_tol=1e-2`, `lgrn_tol=1e-2`, `ctrn_tol=1e-2`, `prev=10`.

Initial guess. The initial level set function ϕ^0 is constructed using two arrays: `centers` and `radii`. The array `centers` contains coordinates that represent centers of balls included in $\mathcal{D}$, and the array `radii` contains their corresponding radii. Then, we define $\phi^0 \colon \overline{\mathcal{D}} \to \mathbb{R}$ as follows:

$$\phi^0(x) = \texttt{factor} \cdot \max_k \phi_k(x), \tag{46}$$

where each function $\phi_k \colon \overline{\mathcal{D}} \to \mathbb{R}$ is given by $\phi_k(x) = \texttt{radii}[k] - |x - \texttt{centers}[k]|_{\texttt{ord}}$. Here, `factor` is a non-zero float number and `ord` is the order of the norm $|\cdot|_{\texttt{ord}}$. By default, `factor=1.0` and `ord=2` (Euclidean norm). Note that the set $\left\{x \in \overline{\mathcal{D}} \mid \texttt{factor} \cdot \phi_k(x) < 0\right\}$ determines either the complement of a closed ball or an open ball, if `factor` is positive or negative, respectively. Starting from (46) and applying level set operations, we see that the initial domain $\Omega^0 = \left\{x \in \overline{\mathcal{D}} \mid \phi^0(x) < 0\right\}$ is given by

$$\Omega^0 = \begin{cases} \overline{\mathcal{D}} \setminus \bigcup_k \overline{\mathbb{B}(k)} & \text{if } \texttt{factor} > 0, \\ \bigcup_k \mathbb{B}(k) & \text{if } \texttt{factor} < 0, \end{cases}$$

where $\mathbb{B}(k)$ is the `ord`-norm open ball centered at `centers`$[k]$ with radius `radii`$[k]$. Thus, Ω^0 can be either $\overline{\mathcal{D}}$ with ball shaped holes or the union of balls included in $\overline{\mathcal{D}}$. For example, after instantiating an object of a subclass of `Model`, we can invoke the method `create_initial_level` to pass the arrays that define the initial level set function ϕ^0: `md.create_initial_level(centers,radii)`, where `md` is a `Model` object.

Dirichlet extension. In order to compute distributed shape derivatives, one sometimes need to extend functions defined on the boundary to the entire domain $\overline{\mathcal{D}}$. This is precisely the case with the boundary measurements g in Section 3. For this purpose, we consider the Dirichlet extension of a function $u \in H^{1/2}(\Gamma_{\text{sub}})^d$, where $\Gamma_{\text{sub}} \subseteq \Gamma$ is a subset of the boundary, defined as the solution to the following problem: Find $\eta \in H^1(\mathcal{D})^d$ such that

$$\begin{aligned} \int_{\mathcal{D}} D\eta \colon D\zeta = 0 \qquad & \forall\, \zeta \in H^1_{\Gamma_{\text{sub}}}(\mathcal{D})^d, \\ \eta = u \qquad & \text{on } \Gamma_{\text{sub}}. \end{aligned} \tag{47}$$

The `dirichlet_extension` function sets up and solve (47) for a list of functions in $H^{1/2}(\Gamma_{\text{sub}})^d$, and returns a list with the corresponding Dirichlet extensions. For instance, by solving (47), we generate the displacement data g used in (13). Since the displacement data must correspond to a force application, we provide the `dir_extension_from` function, which solves (12) for a given set of forces, and then extends the resulting boundary displacement by calling the `dirichlet_extension` function.

Penalized subdomains. In some applications, specific subregions of $\overline{\mathcal{D}}$ must remain fixed. To handle this, we have implemented the `Subdomain` class. This class constructs an indicator function from a list of inequalities that define a subdomain Ω_0 of $\overline{\mathcal{D}}$. We can then use this function to add a penalty term of the form

$$10^4 \int_{\mathcal{D}} (\theta \cdot \xi)\, \chi_{\Omega_0} \tag{48}$$

to the bilinear form B, where 10^4 is a weighting factor. Solving (35) with this additional term enforces $\theta \approx 0$ within Ω_0, effectively keeping the subdomain Ω_0 unperturbed during the shape optimization process. For instance, the subdomain $\Omega_0 = \left\{(x, y) \in \overline{\mathcal{D}} \mid 1.95 < x,\, 0.42 < y < 0.58\right\}$ is modeled using a function that returns the list `[x[0] - 1.95, x[1] - 0.42, 0.58 - x[1]]`. Each element of this list represents an inequality that must be greater than zero. The function must be decorated with `@region_of(domain)`, where `domain` denotes the current domain. This decorator acts as a wrapper for the `Subdomain` class and converts the inequalities into an indicator function. Once defined, the penalty term (48) can be added to the bilinear form by including `B += 1e4self.sbdot(th, xi)*self.dx`, where `self.sb` is the function associated with the list of inequalities.

Computational setup. All numerical experiments were carried out in an `Anaconda` environment using `Python 3.11.10`, together with `FEniCSx 0.9.0` for the finite element computations and `Gmsh 4.12.2` for mesh generation. Other important modules are `NumPy 1.26.3`, `SciPy 1.12.0`, `Matplotlib 3.8.3`, and `MPI4Py 4.0.3`. An installation manual is provided in the `README` file of the following `GitHub` repository: (`github.com/JD26/FormOpt`).

Most of the tests were executed on a laptop; a server was used only when explicitly indicated. Their specifications are as follows:

- **Laptop:** Ubuntu 22.04.5 LTS, equipped with an Intel Core i9-13900H processor (14 physical cores, 20 threads) and 62 GB of RAM.
- **Server:** Debian 12 (bookworm) equipped with two AMD EPYC 9684X processors (192 physical cores in total, 1 thread per core) and 1.5 TB of RAM.

7. Numerical results

7.1. Inverse elasticity

We consider the inverse elasticity problem described in Section 3, where the distributed shape derivative has been computed. We take A_Ω with $\kappa = 10$ in (11). In both examples, the domain $\mathcal{D}$ is a semi-ellipse truncated at the bottom. To account for multiple force-displacement pairs $\{(f_k, g_k)\}_k$, we extend the cost functional (8) as follows:

$$J(\Omega) \coloneqq \frac{\alpha}{2} \sum_k \int_{\mathcal{D}} |u_k - y_k|^2 + \frac{\beta}{2} \sum_k \int_{\Gamma_1} |u_k - y_k|^2, \tag{49}$$

where u_k is the solution to (9) with $f = f_k$, and y_k is the solution to (10) with $g = g_k$. The pairs of applied forces and observed displacements $\{(f_k, g_k)\}_k$ are generated on a finer mesh than that used in the examples, by applying forces on Γ_1 and recording the corresponding displacements. Each g_k is then extended to the entire domain by solving (47). This procedure is performed at the beginning of the example codes (see the `test_6` and `test_34` functions in `test.py`).

In the numerical results, the zero-displacement boundaries are indicated in red, and the true inclusion boundaries are shown in green. In both examples, we employ the bilinear form

$$B(\theta, \xi) \coloneqq \int_{\mathcal{D}} 10^{-1} \theta \cdot \xi + D\theta : D\xi$$

defined for $\theta, \xi \in \mathbb{H} = H_0^1(\mathcal{D})^d$ and the parameter values `niter=200`, `dfactor=0.1`, `lv_time=(1e-3,1.0)`, `cost_tol=0.1`. Implementation details are provided in Appendix B.

Example 1 (Recovery of a single inclusion). The data consists of three pairs of force f_k and displacement g_k. The forces are applied separately on different parts of Γ_1, all directed toward the center of the body. The resulting boundary displacements are measured, only in Γ_1 since Γ_0 is fixed. Thus, we have six state equations and their corresponding adjoints. We conducted three separate experiments: the first using data parallelism with 6 processes, the second using task parallelism with 6 processes, and the third using mixed parallelism with 12 processes (organized into 6 groups of 2 processes each). The execution times were 33, 29, and 27 seconds, respectively. These tests correspond to `test_06`, `test_07`, and `test_08` in the `test.py` file. The results are shown in Figure 2, with the corresponding convergence history presented in Figure 3.

Example 2 (Recovery of two inclusions). The data consists of eight pairs of force f_k and displacement g_k. The forces are applied separately on different parts of Γ_1, all directed toward the center of the body, and covering the whole boundary Γ_1; then resulting boundary displacements are measured. Thus, we have sixteen state equations and their corresponding adjoints. We conducted three separate experiments: the first using data parallelism with 16 processes, the second using task parallelism with 16 processes, and the third using mixed parallelism with 32 processes (organized into 16 groups of 2 processes each). The execution times were 146, 660, and 43 seconds, respectively. These tests were carried out on the server and correspond to `test_34`, `test_35`, and `test_36` in the `test.py` file. See the recovered inclusions in Figure 4.

Figure 2: Inverse elasticity, *Example* 1. Initial and recovered inclusion at iteration $i = 124$ using data parallelism. A mesh with 12 713 triangles was employed. The green line represents the ground truth. The other parallelism modes yield comparable results: 114 and 116 iterations with task and mixed parallelism, respectively.

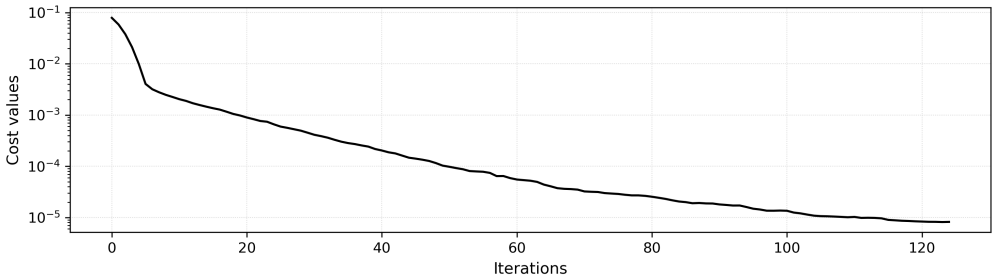


Figure 3: Inverse elasticity, *Example* 1. Convergence of objective function $J(\Omega)$ over iterations using data parallelism.

Figure 4: Inverse elasticity, *Example* 2. Initial and recovered inclusions at iteration $i = 174$ using data parallelism. A mesh with 13 391 triangles was employed. The green line represents the ground truth. The other parallelism modes yield comparable results: 173 and 174 iterations with task and mixed parallelism, respectively.

7.2. Compliance minimization

Compliance minimization is a standard problem in structural mechanics, for which an abundant literature exists, see [8, 9, 4, 11, 20, 6]. Compliance minimization using the distributed shape derivative is also the main topic of the previous work [17]. We use it here to illustrate the performance of `FormOpt` on benchmark problems. Instead of working directly with the variable domain Ω, it is convenient to reformulate the problem on the fixed domain $\mathcal{D}$. To this end, we follow the standard approach of approximating the original problem by filling the complement $\overline{\mathcal{D}} \setminus \Omega$ with an "ersatz material".

We minimize the compliance in a linear elasticity problem:

$$\min_{\Omega \subset \mathcal{D}} J(\Omega) \coloneqq \int_{\mathcal{D}} A_\Omega \sigma(u) : \epsilon(u) \tag{50}$$

subject to

$$\int_{\mathcal{D}} \chi_\Omega = V, \tag{51}$$

where u is the solution to the variational formulation: Find $u \in H^1_{\Gamma_0}(\mathcal{D})^d$ such that

$$\int_{\mathcal{D}} A_\Omega \sigma(u) \colon \epsilon(v) = \int_{\Gamma_1} g \cdot v \qquad \forall\, v \in H^1_{\Gamma_0}(\mathcal{D})^d, \tag{52}$$

and $A_\Omega \colon \overline{\mathcal{D}} \to \mathbb{R}$ is given by $A_\Omega = \chi_\Omega + 10^{-4}\chi_{\overline{\mathcal{D}}\setminus\Omega}$. The corresponding strong formulation is the following transmission problem:

$$\begin{aligned}
-\operatorname{div} A_\Omega \sigma(u) &= 0 && \text{in } \Omega \text{ and } \overline{\mathcal{D}} \setminus \Omega \\
u &= 0 && \text{on } \Gamma_0 \\
A_\Omega \sigma(u) n &= g && \text{on } \Gamma_1 \\
A_\Omega \sigma(u) n &= 0 && \text{on } \Gamma \setminus (\Gamma_0 \cup \Gamma_1) \\
(A_\Omega \sigma(u) n)^+ &= (A_\Omega \sigma(u) n)^- && \text{on } \partial\Omega \\
u^+ &= u^- && \text{on } \partial\Omega.
\end{aligned} \tag{53}$$

The solution to the adjoint problem is $p = -2u$. From (51), the constraint function is given by

$$C(\Omega) = \frac{1}{V} \int_{\mathcal{D}} \chi_\Omega.$$

The derivative components of the compliance $J(\Omega)$ and the constraint function $C(\Omega)$ are

$$\begin{aligned}
S_0^J &= \mathbf{0}, \quad S_1^J = A_\Omega(2Du^\top \sigma(u) - \sigma(u) : \epsilon(u))I, \\
S_0^C &= \mathbf{0}, \quad S_1^C = \frac{1}{V}\chi_\Omega I.
\end{aligned}$$

The `Compliance` and `CompliancePlus` classes contain the equations for compliance minimization. In `CompliancePlus`, we extend `Compliance` by considering multiple forces $\{g_k\}_k$, along with the cost functional $J(\Omega)$ as a sum of compliances associated to each g_k, namely

$$J(\Omega) \coloneqq \sum_k \int_{\mathcal{D}} A_\Omega \sigma(u_k) : \epsilon(u_k), \tag{54}$$

where u_k is the solution to (52) with $g = g_k$.

In the result plots of the following examples, the region with zero displacement and the force application areas are shown in red and blue, respectively.

Example 3 (Symmetric cantilever). We consider the rectangular domain $\mathcal{D} = (0, 2) \times (0, 1)$ with boundary subsets $\Gamma_0 = \{0\} \times (0, 1)$ and $\Gamma_1 = \{2\} \times (0.45, 0.55)$. The vertical force $g = (0, -2)^\top$ is applied on Γ_1 and the material area is constrained to $V = 1$. We run this example using data parallelism with four processes, resulting in an execution time of 6 seconds. This example corresponds to `test_01` in the `test.py` file.

We have performed a reinitialization of the level set function every four steps, with 20 iterations and a final time of 0.1. This yields a well approximated distance function. The other parameters passed to the `runDP`

method have the following values (with default values used for all unspecified parameters): `ctrn_tol=1e-3`, `dfactor=1e-1`, `reinit_step=4`, `reinit_pars=(20,0.1)`, `smooth=True`.

Note that smoothness of the level set function was enforced by setting the parameter `smooth=True`. The bilinear form used in this example is

$$B(\theta,\xi) \coloneqq \int_{\mathcal{D}} 10^{-1}\theta\cdot\xi + D\theta : D\xi + 10^{4}(\theta\cdot\xi)\chi_{\Omega_0} + \int_{\partial\mathcal{D}} 10^{4}(\theta\cdot n)(\xi\cdot n), \tag{55}$$

with $\theta,\xi \in \mathbb{H} = H^1(\mathcal{D})^d$. Thus, B allows only tangential displacements along the boundary $\partial\mathcal{D}$, and penalizes the material around the force application boundary through the term $10^4(\theta\cdot\xi)\chi_{\Omega_0}$, where the subdomain $\Omega_0 \coloneqq (1.95, 2] \times (0.42, 0.58) \subset \overline{\mathcal{D}}$ contains the boundary Γ_1. The results are presented in Figures 5 and 6. The convergence history is consistent with standard results in the literature; see for instance [9, 55].

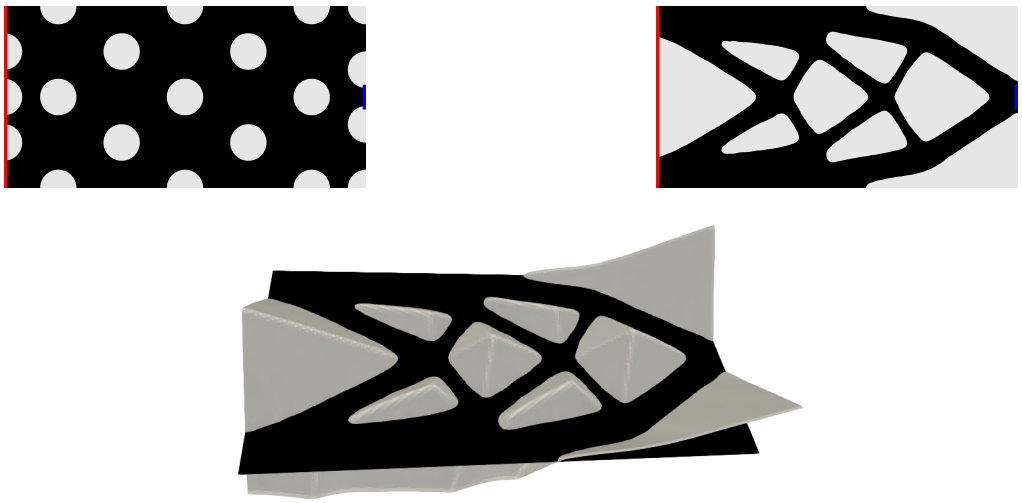


Figure 5: Compliance minimization, *Example* 3. Initial guess (top-left) and optimized design (top-right) at iteration $i = 50$. The resulting level set function ϕ^i approximates the distance function associated to Ω^i (bottom). The domain $\mathcal{D}$ was discretized with 20 946 triangles. Γ_0 appears in red and Γ_1 in blue.

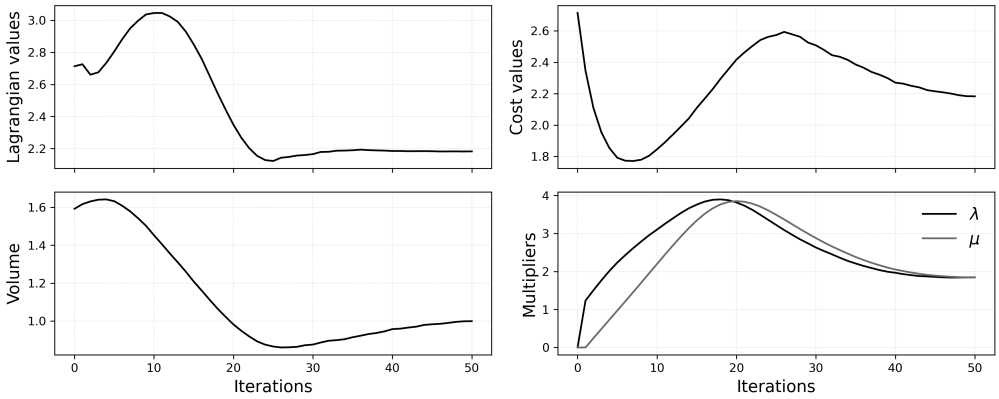


Figure 6: Compliance minimization, *Example* 3. Convergence history of the Lagrangian functional, cost function, constraint function, and Lagrange multipliers.

Example 4 (Three-dimensional cantilever). We consider the rectangular box domain $\mathcal{D} = (0,2) \times (0,1) \times (0,1)$ with boundary subsets $\Gamma_0 = \{0\} \times (0,1) \times (0,1)$ and $\Gamma_1 = \{1\} \times (0.4, 0.6) \times (0.4, 0.6)$. The force $g = (0, 0, -4)^\top$ is applied on Γ_1, and volume material is constrained to $V = 1$. We run this example using data parallelism with 1, 2, and 4 processes, resulting in executions times of 4.357, 2.332, and 1.486 hours,

respectively. We employ the bilinear form (55) with the subdomain

$$\Omega_0 = (1.90, 2] \times (0.35, 0.65) \times (0.35, 0.65)$$

in order to penalize the material around Γ_1. The level set function used as initial guess was constructed with infinity-norm balls as holes (i.e., by setting `ord=np.inf` in the `create_initial_level` function). This choice is consistent with the the regular cubic lattice of the mesh generated by the `create_box` function of `FEniCSX`. The parameters passed to the `runDP` method have the following values: `ctrn_tol=1e-3`, `dfactor=1e-1`, `reinit_step=4`, `reinit_pars=(4,1e-2)`, `smooth=True`. This example corresponds to `test_02` in the `test.py` file; see some iterations in Figure 7.

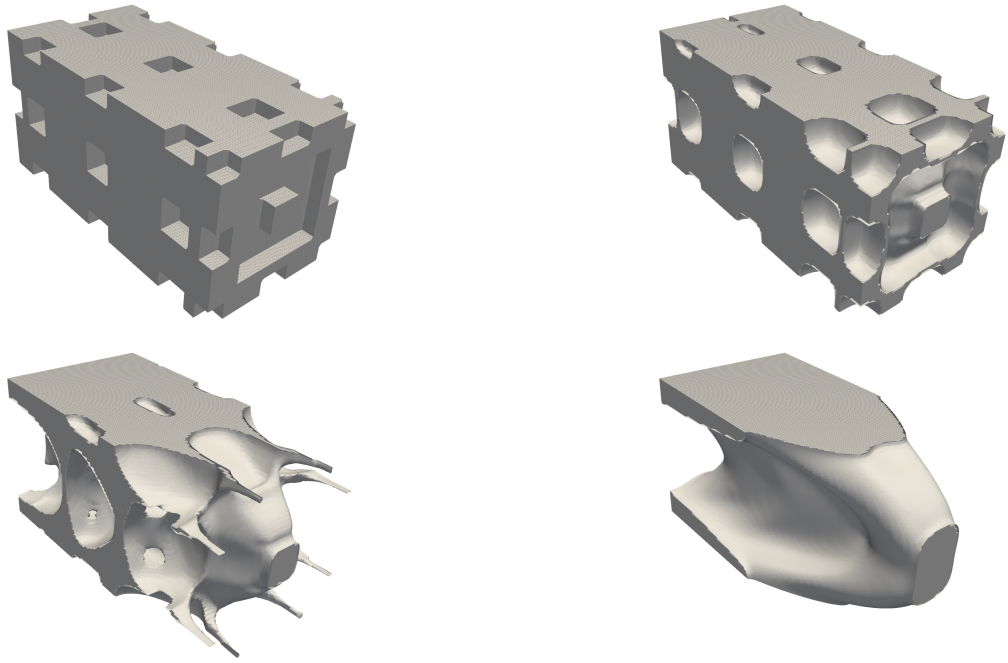


Figure 7: Compliance minimization, *Example* 4. The three-dimensional cantilever at $i = 0$ and $i = 14$ (top); $i = 30$ and $i = 81$ (bottom). A mesh with 2 592 000 tetrahedra was employed.

Example 5 (Cantilever with two loads I). Here $\mathcal{D}$ is the unit square and we solve the problem with multiple forces $g^{\mathrm{I}} = g^{\mathrm{II}} = (0, -2)^\top$, applied on $\Gamma_1^{\mathrm{I}} = \{x = 1\} \times (0, 0.1)$ (right-upper side) and $\Gamma_1^{\mathrm{II}} = \{x = 1\} \times (0.9, 1)$ (right-bottom side), respectively. The boundary of zero displacement is $\Gamma_0 = \{x = 0\} \times (0, 1)$, and the material area is constrained to $V = 0.5$. Thus, we minimize the cost functional (54), which is implemented in the `cost` method of the `CompliancePlus` class. To compare performances, we conducted three experiments: the first using data parallelism with 2 processes, the second using task parallelism with 2 processes, and the third using mixed parallelism with 4 processes (organized into 2 groups of 2 processes each). The execution times were 20, 26, and 15 seconds, respectively. The optimized design and the number of iterations are the same in all cases. We have used the bilinear form (55) without any penalized subdomain. The configuration passed to the `runDP`, `runTP`, and `runMP` methods was: `ctrn_tol=1e-3`, `dfactor=1e-1`, `reinit_step=4`, `reinit_pars=(16,0.05)`, `smooth=True`. The tests in this example correspond to `test_03`, `test_04`, and `test_05` in the `test.py` file. See the results in Figure 8.

Example 6 (Cantilever with two loads II). This example also applies multiple loads. We consider the rectangular domain $\mathcal{D} = (0, 2) \times (0, 1)$. The forces $g^{\mathrm{I}} = (0, -2)^\top$ and $g^{\mathrm{II}} = (0, 2)^\top$ are applied separately on the boundaries $\Gamma_1^{\mathrm{I}} = (0.95, 1.05) \times \{y = 0\}$ (bottom-center side) and $\Gamma_1^{\mathrm{II}} = \{x = 2\} \times (0.45, 0.55)$ (left-center side), respectively. The boundary of zero displacement is $\Gamma_0 = \{x = 0\} \times (0, 1)$ and the area constraint is $V = 1.1$. The performance of data and task parallelisms are compared using the same parameter values, on a mesh of

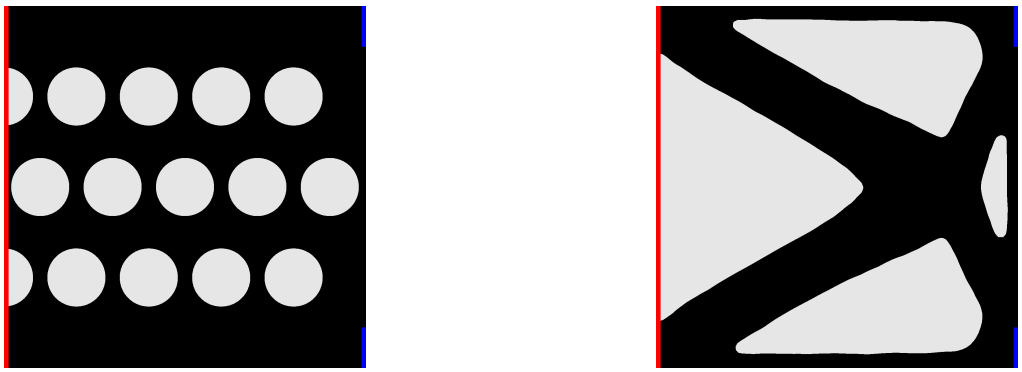


Figure 8: Compliance minimization, *Example* 5. Initial and optimal domains. A total of 70 iterations were performed for the three modes of parallelism. A mesh with 16 425 triangles was employed.

52 155 triangles: `ctrn_tol=1e-3`, `lgrn_tol=1e-3`, `dfactor=1e-1`, `reinit_step=4`, `reinit_pars=(20,1e-2)`, `smooth=True`. The execution times were 70 seconds for data parallelism and 98 seconds for task parallelism. The bilinear form (55) is employed, with the subdomain

$$\Omega_0 \coloneqq (0.94, 1.06) \times [0, 0.05) \cup (1.95, 2] \times (0.42, 0.58)$$

to penalize the material around Γ_1^{I} and Γ_1^{II}. Recall that in task parallelism the number of processes must be equal to the number of state/adjoint problems (2 in this example). See the result in Figure 9.

Figure 9: Compliance minimization, *Example* 6. Initial and optimal domains. Both parallelism modalities require 79 iterations.

7.3. Heat conduction

Shape and topology optimization play a crucial role for efficient thermal management in modern applications such as battery thermal regulation [56], heat exchangers [57], additive-manufactured cooling channels [58], and temperature-sensitive components in aerospace systems.

Let $\Gamma_0 \subset \Gamma = \partial\mathcal{D}$ and V be a prescribed volume, smaller than that of $\mathcal{D}$. Consider the minimization problem

$$\min_{\Omega \subset \mathcal{D}} J(\Omega) \coloneqq \int_{\mathcal{D}} A_\Omega |\nabla u|^2 \tag{56}$$

subject to

$$\int_{\mathcal{D}} \chi_\Omega = V, \tag{57}$$

where u is the solution to the following transmission problem

$$\begin{aligned} -\mathrm{div} A_\Omega \nabla u &= f && \text{in } \Omega \text{ and } \mathcal{D} \setminus \overline{\Omega} \\ u &= 0 && \text{on } \Gamma_0 \\ \partial_n u &= 0 && \text{on } \Gamma \setminus \Gamma_0 \\ (A_\Omega \partial_n u)^+ &= (A_\Omega \partial_n u)^- && \text{on } \partial\Omega \\ u^+ &= u^- && \text{on } \partial\Omega \end{aligned} \tag{58}$$

and $A_\Omega\colon \overline{\mathcal{D}} \to \mathbb{R}$ is given by $A_\Omega = \chi_\Omega + 10^{-3}\chi_{\overline{\mathcal{D}}\setminus\Omega}$. According to (2), the constraint function for this problem is written as

$$C(\Omega) = \frac{1}{V}\int_{\mathcal{D}} \chi_\Omega. \tag{59}$$

The weak formulation of (58) reads: Find $u \in H^1_{\Gamma_0}(\mathcal{D})$ such that

$$\int_{\mathcal{D}} A_\Omega \nabla u \cdot \nabla v = \int_{\mathcal{D}} fv \quad \forall\, v \in H^1_{\Gamma_0}(\mathcal{D}). \tag{60}$$

In this case, the adjoint problem is the same as (60) (except by a minus sign on the right-hand side), and thus $p = -u$. The shape derivative components are given by

$$S_0^J = 2u\nabla f, \qquad S_1^J = (2uf - A_\Omega|\nabla u|^2)I + 2A_\Omega \nabla u \otimes \nabla u$$

for the cost functional, and $S_0^C = \mathbf{0}$, $S_1^C = \frac{1}{V}\chi_\Omega I$ for the constraint function. In all examples, we use the square domain $\mathcal{D} = (0,1)\times(0,1)$, and red color to highlight Γ_0, where the temperature is fixed to zero.

Example 7. We apply a uniform heat $f \equiv 1$, with volume constraint $V = 0.25$ and $\Gamma_0 = (0.4, 0.6)\times\{y = 0\}$. The test is carried out using data parallelism with two processes and without the diffusion term in the transport equation (31) (i.e., `smooth=False` in the `runDP` function). The execution time is 58 seconds. We have employed the bilinear form

$$B(\theta,\xi) = \int_{\mathcal{D}} D\theta : D\xi + 10^4(\theta\cdot\xi)\chi_{\Omega_0},$$

with $\theta, \xi \in \mathbb{H} = H^1_0(\mathcal{D})^d$ and $\Omega_0 = (0.3, 0.7)\times[0, 0.05)$, in order to penalize the material around Γ_0. This example corresponds to `test_09` in the `test.py` file; see Figures 10 and 11 for the results.

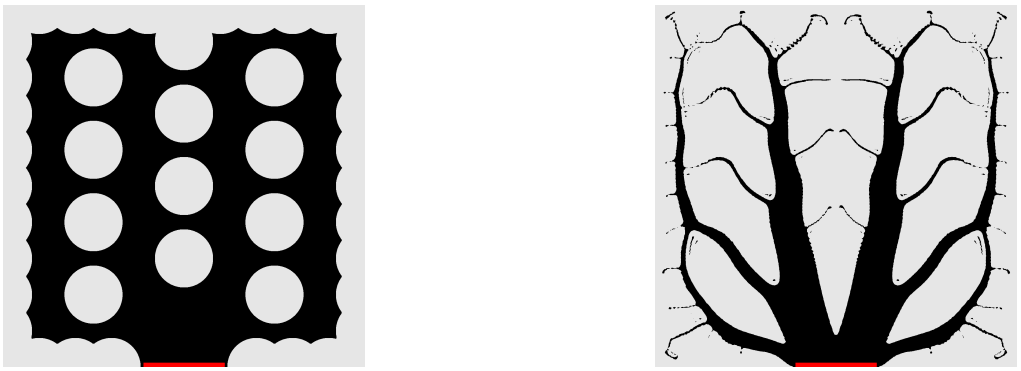


Figure 10: Heat conduction, *Example* 7. Initial guess and optimized design at iteration $i = 204$, computed on a mesh with 92 582 triangles. The black-colored region Ω has the highest conductivity.

Example 8. We consider a uniform heat $f \equiv 1$, a volume constraint $V = 0.6$, and $\Gamma_0 = \Gamma$ (i.e., the whole boundary is kept at zero temperature). We run this example using data parallelism with four processes, resulting in an execution time of 52 seconds. No diffusion term is included in the transport equation (31). This example corresponds to `test_10` in the `test.py` file; see the results in Figure 12. The bilinear form B is the same as in *Example* 7, but with Ω_0 defined as

$$\Omega_0 \coloneqq \{x < 0.1\} \cup \{x > 0.9\} \cup \{y < 0.1\} \cup \{y > 0.9\}.$$

Thus, the material is penalized close to the boundary. Notice the typical fractal structure of the optimized geometries in Figures 10 and 12; compare the results with those in [57].

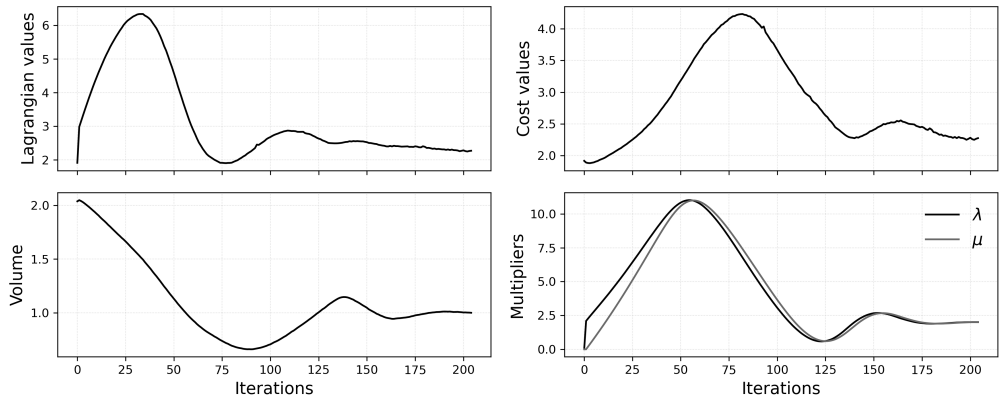


Figure 11: Heat conduction, *Example* 7. Convergence history of the Lagrangian functional L (see (32)), cost function $J(\Omega)$, constraint function $C(\Omega)$, and Lagrange multipliers λ, μ.

Figure 12: Heat conduction, *Example* 8. Initial guess and optimized design at iteration $i = 172$, computed on a mesh with 144 712 triangles. The black-colored region Ω has the highest conductivity.

Example 9. We apply a localized heat source given by the function $f(x,y) \coloneqq \omega(|(x,y)-(0.5,0.5)|_2)$, where ω is the C^1 radial function defined as

$$\omega(r) \coloneqq \begin{cases} 25(1+\cos(10\pi r)) & \text{if } r < 0.1, \\ 0 & \text{otherwise.} \end{cases} \tag{61}$$

Thus, f is radially symmetric around the point $(0.5, 0.5)$ and C^1 on the entire plane. The support of f is the disk centered at $(0.5, 0.5)$ with radius 0.1. The volume constraint is $V = 0.5$ and $\Gamma_0 = \Gamma$. We run this example using data parallelism with six processes, resulting in an execution time of 11 seconds. This example corresponds to `test_11` in the `test.py` file; see the results in Figure 13. The term (36) is added to the bilinear form $B(\theta,\xi) = \int_{\mathcal{D}} D\theta : D\xi$ and the velocity problem (35) is solved in $\mathbb{H} = H^1(\mathcal{D})^d$, allowing tangential displacements along the boundary $\partial\mathcal{D}$.

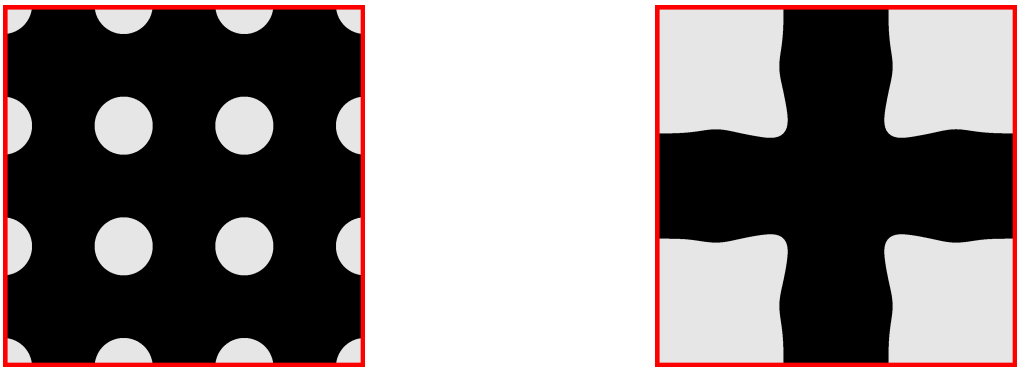


Figure 13: Heat conduction, *Example* 9. Initial guess and optimized design at iteration $i = 181$, computed on a mesh with 23 652 triangles.

Example 10. In this example, we perform two tests. In the first test, we solve the problem with a single heat sink $\Gamma_0 = \Gamma_0^{\mathrm{I}} \cup \Gamma_0^{\mathrm{II}}$, where $\Gamma_0^{\mathrm{I}} = (0.4, 0.6) \times \{y = 0\}$, $\Gamma_0^{\mathrm{II}} = \{x = 1\} \times (0.4, 0.6)$. In the second test, we solve the problem with multiple heat sinks, namely Γ_0^{I} and Γ_0^{II}. In this case, the cost functional is defined as the sum of the corresponding thermal compliances:

$$J(\Omega) \coloneqq \frac{1}{2}\int_{\mathcal{D}} A_\Omega \left|\nabla u^{\mathrm{I}}\right|^2 + \frac{1}{2}\int_{\mathcal{D}} A_\Omega \left|\nabla u^{\mathrm{II}}\right|^2,$$

where u^{I} solves (58) with $\Gamma_0 = \Gamma_0^{\mathrm{I}}$ and u^{II} solves (58) with $\Gamma_0 = \Gamma_0^{\mathrm{II}}$. In both tests, $f \equiv 1$ and $V = 0.4$. The first test is carried out using data parallelism with two processes, resulting in an execution time of 68 seconds. Since the second test involves solving two PDEs, we employ task parallelism with two processes, resulting in an execution time of 106 seconds. These tests correspond to `test_12` and `test_13` in the `test.py` file. The results are shown in Figure 14.

Example 11. As in the previous example, here we consider the single and multiple cases, but for the heat source. In the single case test, we apply one heat source $f = f^{\mathrm{I}} + f^{\mathrm{II}} + f^{\mathrm{III}} + f^{\mathrm{IV}}$, where

$$f^{\mathrm{I}}(x,y) \coloneqq \omega(|(x,y)-(0.5,0.25)|_2), \qquad f^{\mathrm{II}}(x,y) \coloneqq \omega(|(x,y)-(0.75,0.5)|_2),$$
$$f^{\mathrm{III}}(x,y) \coloneqq \omega(|(x,y)-(0.5,0.75)|_2), \qquad f^{\mathrm{IV}}(x,y) \coloneqq \omega(|(x,y)-(0.25,0.5)|_2).$$

The function ω was defined in (61). In the multiple case, we consider four heat sources $f = f^{\mathrm{I}}$, $f = f^{\mathrm{II}}$, $f = f^{\mathrm{III}}$, and $f = f^{\mathrm{IV}}$, along with the cost functional

$$J(\Omega) \coloneqq \sum_{i=\mathrm{I}}^{\mathrm{IV}} \frac{1}{4}\int_{\mathcal{D}} A_\Omega \left|\nabla u^i\right|^2,$$

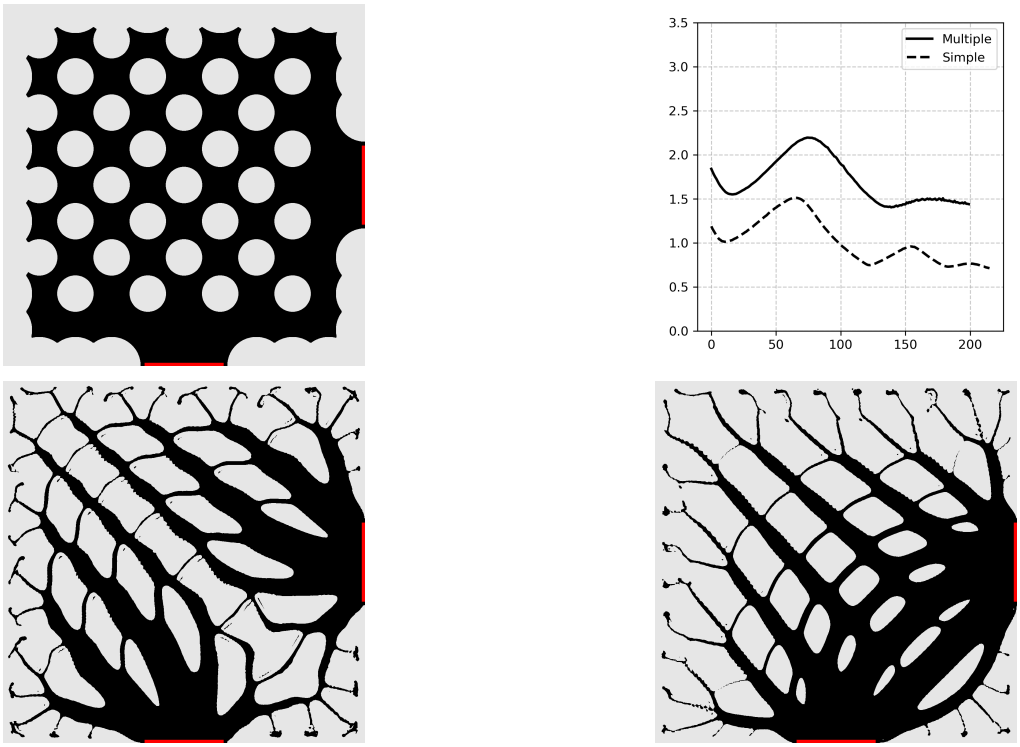


Figure 14: Heat conduction, *Example* 10. Initial guess (top-left) and cost values (top-right), along with the optimized designs of the single heat sink (bottom-left) and multiple heat sink (bottom-right), at iterations $i = 215$ and $i = 199$, respectively. The final cost value of the multiple sink test approximately duplicates the one of the single sink test.

where u^i solves (58) with $f = f^i$. In both cases, the volume constraint is $V = 0.45$, and Γ_0 consists of four small subsets of the boundary in the corners of the domain, each one with length $0.05\sqrt{2}$. The first test is carried out using data parallelism with two processes, resulting in an execution time of 15 seconds. Since in the second test there are four PDEs to be solved, we employ task parallelism with four processes, resulting in an execution time of 20 seconds. These tests correspond to `test_21` and `test_22` in the `test.py` file. The numerical results are shown in Figure 15; they agree with the results reported in [59], which we are able to reproduce within our framework.

7.4. A nonlinear PDE arising in population dynamics

We present an example in which the PDE constraint is nonlinear. Consider the problem of spatially distributing resources in order to maximize the population size of a species that consumes those resources. We assume that the population growth is governed by the logistic equation with a diffusive term. Since the temporal derivative is absent, the objective is to maximize the population size in the long-time regime, when the species distribution has reached a stationary state. In this setting, the PDE constraint takes the form of a nonlinear diffusion equation [60].

Let V be a prescribed volume, smaller than the volume of $\mathcal{D}$. The problem reads

$$\min_{\Omega \subset \mathcal{D}} J(\Omega) \coloneqq -\int_{\mathcal{D}} u \tag{62}$$

subject to $\int_{\mathcal{D}} \chi_\Omega = V$, where u is the solution to

$$\begin{aligned} -\Delta u &= ru\left(1 - \frac{u}{K_\Omega}\right) && \text{in } \Omega \text{ and } \mathcal{D} \setminus \overline{\Omega} \\ \partial_n u &= 0 && \text{on } \Gamma \\ (\partial_n u)^+ &= (\partial_n u)^- && \text{on } \partial\Omega \\ u^+ &= u^- && \text{on } \partial\Omega \end{aligned} \tag{63}$$

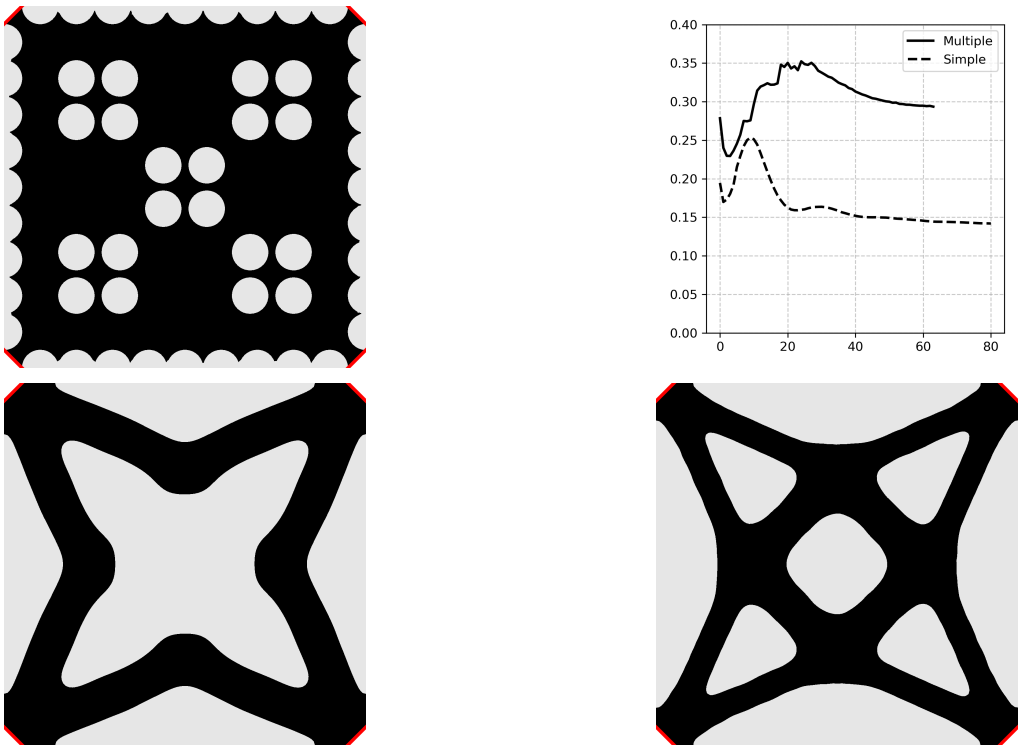


Figure 15: Heat conduction, *Example* 11. Initial guess (top-left) and cost values (top-right), along with the optimized designs of the single case (bottom-left) and multiple case (bottom-right), at iterations $i = 80$ and $i = 63$, respectively. The final cost value of the multiple source test approximately duplicates the one of the single source test.

and $K_\Omega \colon \overline{\mathcal{D}} \to \mathbb{R}$ is given by

$$K_\Omega = \chi_\Omega + 10^{-2}\chi_{\overline{\mathcal{D}}\setminus\Omega}. \tag{64}$$

According to (2), the constraint function for this problem is written as

$$C(\Omega) = \frac{1}{V}\int_{\mathcal{D}} \chi_\Omega. \tag{65}$$

Here, u represents the equilibrium population density, r is a positive constant that represents the growth rate of the population in the logistic model, and the positive function K_Ω represents the spatially varying carrying capacity, i.e., the maximum equilibrium population density that the available resources can sustain at each point. Equation (64) indicates the presence of two types of resources, with one permitting significantly higher growth than the other. Moreover, we constrain the distribution of the main resource to be supported in a region of volume V.

The weak formulation of (63) is the following: Find $u \in H^1(\mathcal{D})$ such that

$$\int_{\mathcal{D}} \nabla u \cdot \nabla v = \int_{\mathcal{D}} ru\left(1 - \frac{u}{K_\Omega}\right) v \quad \forall\, v \in H^1(\mathcal{D}). \tag{66}$$

From the Lagrangian functional, which is constructed as in Section 3, we obtain the adjoint equation: Find $p \in H^1(\mathcal{D})$ such that

$$\int_{\mathcal{D}} \nabla p \cdot \nabla q + \int_{\mathcal{D}} r\left(\frac{2u}{K_\Omega} - 1\right) pq = \int_{\mathcal{D}} q \quad \forall\, q \in H^1(\mathcal{D}).$$

Observe that although the state problem is nonlinear, the adjoint problem is linear. The derivative components of the cost functional (62) and the constraint (65) are

$$S_0^J = \mathbf{0}, \qquad S_1^J = \left(\nabla u \cdot \nabla p - u - ru\left(1 - \frac{u}{K_\Omega}\right) p\right) I - (\nabla u \otimes \nabla p + \nabla p \otimes \nabla u),$$

and $S_0^C = \mathbf{0}$, $\quad S_1^C = \frac{1}{V}\chi_\Omega I$.

Example 12. We solve problem (62)–(63) on the unit square $\mathcal{D} = (0,1) \times (0,1)$ with a resource constraint $V = 0.5$. The bilinear form used in this problem is

$$B(\theta, \xi) := \int_{\mathcal{D}} D\theta : D\xi + 10^4 \int_{\partial\mathcal{D}} (\theta \cdot n)(\xi \cdot n),$$

with $\theta, \xi \in \mathbb{H} = H^1(\mathcal{D})$. Thus, we allow the resource to move freely along the boundary $\partial\mathcal{D}$. We perform tests with growth rates $r = 10$, 40, and 100, all using two processes, resulting in execution times of 547, 321, and 235 seconds, respectively. These tests correspond to `test_14`, `test_15`, and `test_16` in the `test.py` file. The optimized resource distributions are shown in Figure 16, while Figure 17 shows the values of the cost functional.

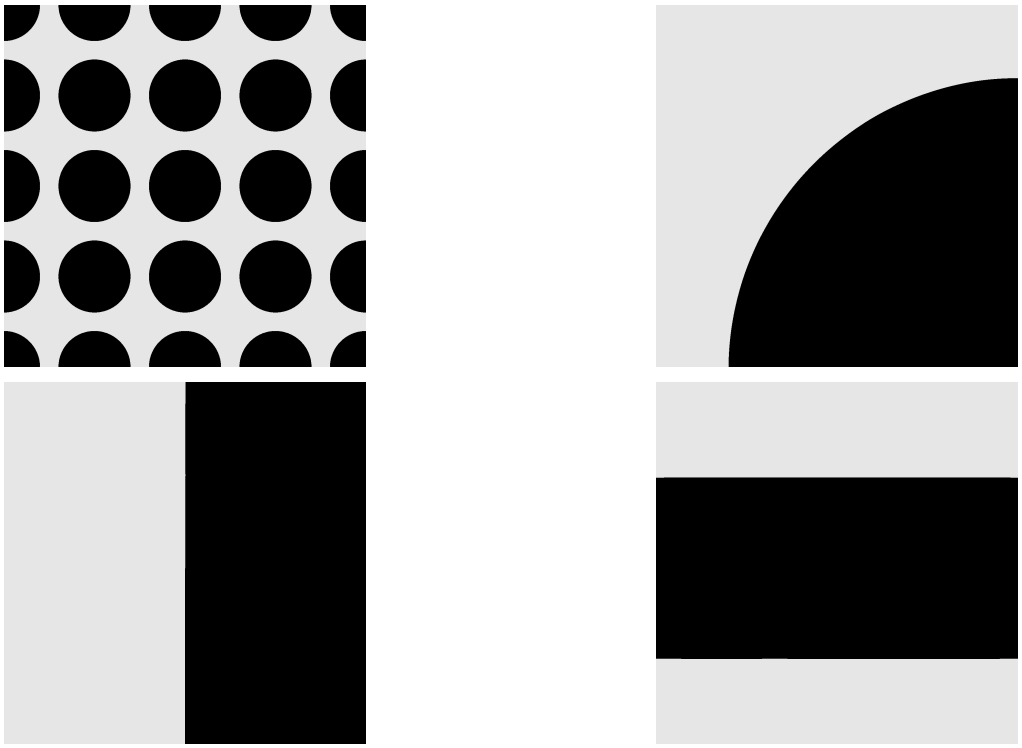


Figure 16: Examples of resource allocation optimization in population dynamics. Initial guess (top-left) and optimized designs for $r = 10$ (top-right), $r = 40$ (bottom-left), and $r = 100$ (bottom-right). The stopping criterion was satisfied at 147, 107, and 74 iterations, respectively.

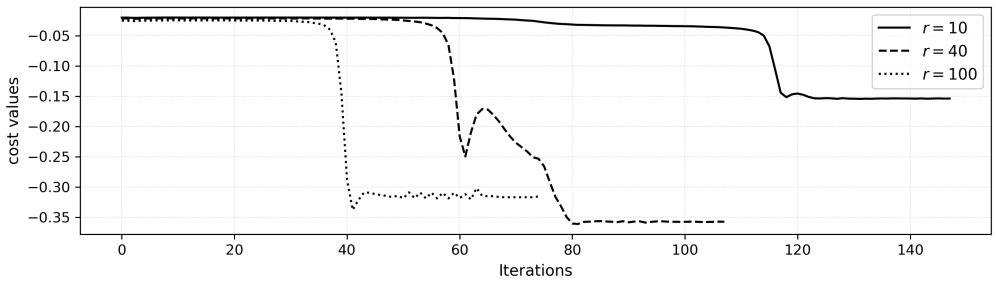


Figure 17: Cost functional history for different growth rates. The curve corresponding to the growth rate $r = 40$ reaches a larger population size.

Newton's method was employed to solve the state equation. For this purpose, the `pde` method returns several related objects: equation (66) in the form $F(u) = 0$, an empty list `[]` indicating the absence of

Dirichlet boundary conditions, the Jacobian $DF(u)(\delta u)$, the `UFL` variable representing the state u, and the positive function $u_0(x, y) = 1 + 0.2\sin(6\pi x)\sin(6\pi y)$ as initial guess. The Jacobian is given by

$$DF(u)(\delta u)(v) = \int_{\mathcal{D}} \nabla \delta u \cdot \nabla v + \int_{\mathcal{D}} r \left(\frac{2u}{K_\Omega} - 1 \right) \delta u \, v,$$

where δu and v must be represented by a `TrialFunction` and a `TestFunction`, respectively, while the state variable u is treated as a `Coefficient` (rather than a `TrialFunction`, as in linear problems). This setting is required by the Newton solver of `FEniCSX`. The maximum number of iterations and additional configurations can be specified in the `create_nonlinear_solver` function.

The initial guess is a callable function that will be interpolated on the domain mesh. It is provided when a model of the `Logistic` class is created; see the initializer of this class. Our choice of u_0 is motivated by the fact that u represents a population density; hence, it must be positive and cannot exceed the maximum value of K_Ω.

7.5. Performance investigation

Parallel scalability: We present two performance tests aimed at assessing parallel scalability. We revisit the symmetric cantilevers from *Example 1* and *Example 3* in Subsection 7.2, with modified initial conditions to produce different optimization outcomes. Both tests were run on the server using meshes with more than a 200% increase in the number of finite elements compared with the settings of *Example* 3 and *Example* 5 shown in Figure 5 and Figure 8. The finer discretization allows for a greater number of smaller holes in the initial shapes, in contrast to the coarser examples discussed earlier. The results are shown in Figures 18 and 19. The execution time decreased monotonically with the number of processes, up to 12 processes. An almost linear speed-up was observed between 1 and 5 processes, with a pronounced reduction in execution time, followed by a slower, more gradual decrease at higher process counts. Optimized designs with thinner structural parts were obtained.

Figure 18: Performance results on a mesh with 52 085 triangles for the cantilever with one load. Number of processes versus execution times in seconds (left) and optimized design at iteration $i = 45$ (right).

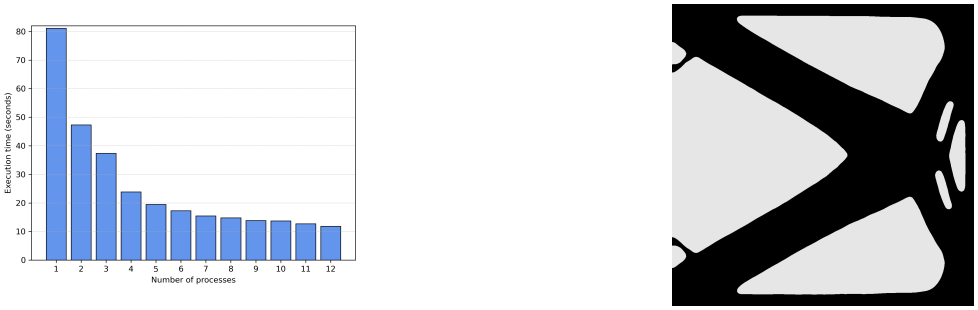


Figure 19: Performance results on a mesh with 36 375 triangles for the cantilever with two loads. Number of processes versus execution times in seconds (left) and optimized design at iteration $i = 61$ (right).

Dependence on bilinear form B: The following test is aimed at comparing the convergence behavior for different values of the weight multiplying the L^2 inner product in the bilinear form B used to compute descent

directions, see (29). We plot the values of the Proximal-Perturbed Lagrangian (32) for the problem presented in Example 3 (compliance minimization for a symmetric cantilever), considering the bilinear form

$$B_a(\theta,\xi) \coloneqq a\int_{\mathcal{D}} \theta\cdot\xi + \int_{\mathcal{D}} D\theta : D\xi + 10^4(\theta\cdot\xi)\chi_{\Omega_0} + \int_{\partial\mathcal{D}} 10^4(\theta\cdot n)(\xi\cdot n), \tag{67}$$

for several values of $a > 0$; see Figure 20. The results show similar behavior for all values of a, except for an increasing number of iterations as a becomes larger. For $a = 100$, 75 iterations were required, which is 50% more than for $a = 0.1$ (the value used in Example 3). This test can be found in the `Examples.ipynb` file.

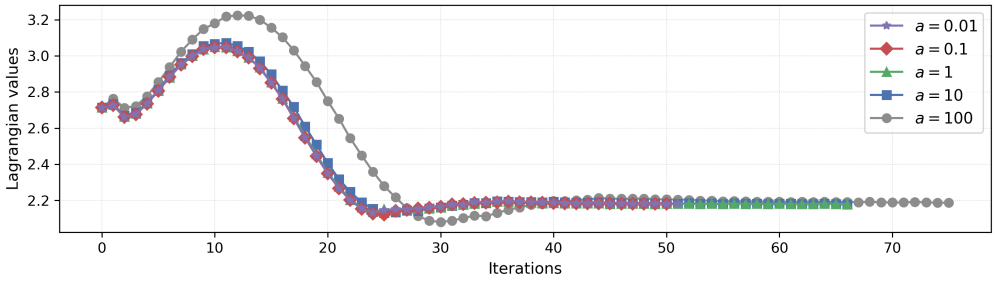


Figure 20: Convergence history of the Lagrangian functional for different values of the weight a in the bilinear form (67).

Dependence on ersatz material parameter: We investigate the performance of the solver for different values of the ersatz material parameter, for the problem of Example 3. We solve this problem for different values of ε in $A_\Omega = \chi_\Omega + \varepsilon\chi_{\overline{\mathcal{D}}\setminus\Omega}$. At each iteration of the level set algorithm (Algorithm 1), we solve the state equation (52) using a PETSc-based GMRES iterative solver with a Hypre multigrid preconditioner. We set the relative and absolute stopping tolerances to `ksp_rtol=1e-6` and `ksp_atol=1e-10`, respectively. Thus, the solver stops when $|r_k| \leq \max\{10^{-6}|r_0|, 10^{-10}\}$, where $r_k = Ax^k - b$ is the residual. We report the number of iterations performed by the iterative solver and the mean of the corresponding reduction factors ρ, where $r_{k+1} = \rho r_k$; see Figure 21. We observe that the solver exhibits similar performance across all values of ε, with a comparable number of iterations required to meet the stopping criterion. A deviation is observed for iterations $i = 15, \ldots, 23$, likely due to the formation of thin structures and cusps, as illustrated in Figure 22. Note that more iterations were required when the residual decreased more slowly. For instance, for $\varepsilon = 10^{-4}$, at iteration $i = 18$, the solver perform 56 iterations, and the mean reduction factor was $r_{k+1} = 80\%r_k$, while at iteration $i = 40$, only 15 solver iterations were necessary and the mean reduction factor was $r_{k+1} = 41\%r_k$.

7.6. Toolbox comparison

We conclude with a succinct comparison of recent open-source toolboxes for shape and topology optimization. The results are summarized in Tables 2, 3, and 4. Given the large and growing number of available tools, this list is not exhaustive but aims to provide a representative overview of recent contributions. Our focus is primarily on methods based on shape and topological derivatives with parallel capabilities. We include [17] as a reference point, as the present work can be viewed as an extension of this contribution. All the toolboxes considered report both two- and three-dimensional results, with the exception of [22], which presents only three-dimensional examples, and [17], which is restricted to two dimensions. As shown in Table 2, most of the toolboxes considered here that evolve the level set via a Hamilton–Jacobi equation rely on finite difference schemes, which constitute the standard approach for solving such hyperbolic problems, while finite elements are employed to solve the state and adjoint equations. In contrast, the `FormOpt` toolbox adopts a finite element formulation throughout, using it for both the Hamilton–Jacobi equation and the state and adjoint problems. This unified approach offers greater flexibility, particularly for handling complex geometries of the hold-all domain $\mathcal{D}$. As shown in Table 2, toolboxes based on level set methods typically rely on a fixed background mesh, with the notable exceptions of Autofreefem [61] and [62], which employ a level set-based mesh evolution strategy.

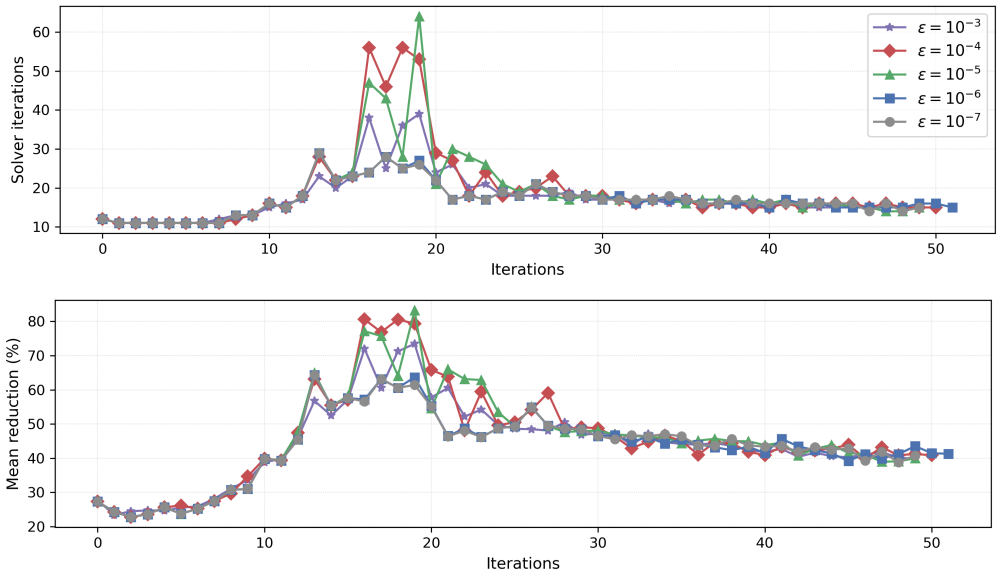


Figure 21: Number of iterations performed by the iterative solver dedicated to the state equation (top) and the means of the corresponding reduction rates of the residuals (bottom) for different values of the ersatz material, both versus the iterations of the level set algorithm.

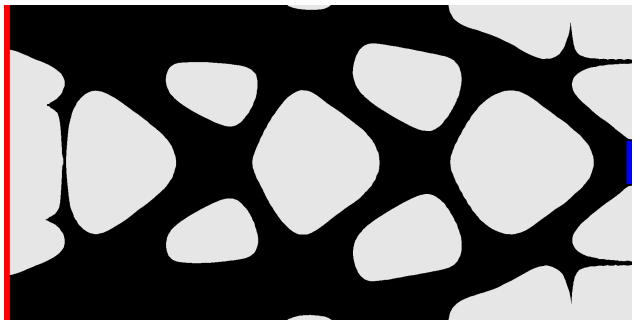


Figure 22: Shape at iteration $i = 18$, exhibiting thin structures and cusps, obtained with an ersatz material parameter $\varepsilon = 10^{-4}$.

Toolbox	Library	Domain evolution	Sensitivity	Physics
FormOpt	FEniCSx	Level set + HJFE	DSD / Analytic	MP
Laurain (2018) [17]	FEniCS	Level set + HJFD	DSD / Analytic	LE
Filho et al. (2025) [22]	FreeFEM, PETSc	Level set	TD / Analytic	LE
OpenLSTO [55]	C++	Level set + HJFD	Discrete adjoint	LE
Li et al. (2021) [62]	FreeFEM, PETSc, Mmg	Level set + RDE	TD / Analytic	LE
GridapTopOpt.jl [30]	Gridap (Julia)	Level set + HJFD + SH	BSD / AD	LE, NLE
Autofreefem [61]	FreeFEM, SymPy	Level set based mesh evolution	SD / Symbolic	MP
cashocs 2.0 [16, 33]	FEniCS	Mesh deformation	SD /AD	MP
Fireshape [18]	Firedrake, ROL	Mesh deformation	ASD	MP
Gangl et al. [14]	NGSolve	Mesh deformation	ASD / semi-ASD	MP
FEMorph [63]	FEniCS, UFL	Mesh deformation	BSD / DSD / ASD	MP
FEniTop [32]	FEniCSx	Density	SIMP / AD	LE
Aage et al. [31]	PETSc	Density	SIMP	LE
Null Space Optimizer [19]	Python	Density	SIMP	MP

Table 2: Comparison of open-source toolboxes for shape and topology optimization. Here SH = smooth Heaviside approach, HJFE = Hamilton-Jacobi, Finite Element scheme, HJFD = Hamilton-Jacobi, Finite Difference scheme, RDE = reaction-diffusion equation, SD = Shape Derivative, DSD = Distributed Shape Derivative, BSD = Boundary Shape Derivative, TD = Topological Derivative, AD = Automatic Differentiation, ASD = Automatic Shape Differentiation, LE = Linear Elasticity, NLE = Nonlinear Elasticity, MP = Multiphysics.

Toolbox	Parallelism	Constraint handling
FormOpt	Native MPI / mixed parallelism	Proximal-Perturbed Lagrangian
Laurain (2018) [17]	Not emphasized	Penalization
Filho et al. (2025) [22]	Native MPI	Augmented Lagrangian
OpenLSTO [55]	Not emphasized	MMA
Li et al. (2021) [62]	Native MPI	Augmented Lagrangian
GridapTopOpt.jl [30]	MPI.jl / GridapDistributed	Augmented Lagrangian / Hilbertian projection method
Autofreefem [61]	Not emphasized	Penalization
cashocs 2.0 [16, 33]	Native MPI	Augmented Lagrangian /quadratic penalty
Fireshape [18]	Native MPI	ROL (augmented Lagrangian / trust-region)
Gangl et al. [14]	Native MPI	PDE constraints
FEMorph [63]	Not emphasized	PDE constraints
FEniTop [32]	Native MPI	MMA / OC
Aage et al. [31]	Native MPI	MMA
Null Space Optimizer [19]	Not emphasized	Projection onto feasible directions and correction flow

Table 3: Comparison of open-source toolboxes for shape and topology optimization. Here MMA = Method of Moving Asymptotes, OC = Optimality Criteria method, ROL = Rapid Optimization Library.

8. Conclusion

This work presents a toolbox for PDE-constrained shape optimization based on the distributed shape derivative and a level-set method. The implementation significantly generalizes the approach of [17]: while inspired by the same underlying concepts, the present implementation is considerably more general in both scope and applicability.

The `FormOpt` toolbox introduces several notable features. The Unified Form Language (UFL) provides an intuitive way to represent weak formulations of PDEs in a notation close to the mathematical one. Our module encapsulates the numerical methods in classes (level set, velocity problem, reinitialization), which can be modified independently, leaving the user with the task of writing the problem equations (weak formulations, cost functional, constraints, derivative components) in UFL format. This modular structure separates the formulation of the equations from the definition of the mesh, finite elements, and boundary conditions. `FormOpt` supports both two- and three-dimensional problems with flexible geometries, enabled by the finite element capabilities of `FEniCSx`. Another key development is the integration of parallel computing with three distinct modes, which significantly enhances efficiency and enables large-scale simulations. In addition, the toolbox includes a built-in Proximal-Perturbed Lagrangian method for handling shape constraints, particularly useful for volume and perimeter constraints, which are ubiquitous in shape optimization.

Several extensions of this work will be pursued in future research. The extension to higher-order tensor representations of distributed shape derivatives is expected to be straightforward and will be valuable for applications, such as problems involving the bi-Laplacian [39]. Advanced discretization strategies based on

Toolbox	Notes & Additional Features
FormOpt	General-purpose shape optimization, FE-based level set method
Laurain (2018) [17]	Distributed shape derivative, educational
Filho et al. (2025) [22]	Topological derivative, parallel framework
OpenLSTO [55]	Hybrid level-set / density formulation
Li et al. (2021) [62]	Mesh adaptivity (Mmg)
GridapTopOpt.jl [30]	automatic differentiation with respect to the level set function
Autofreefem [61]	Automatic code generation
cashocs 2.0 [16, 33]	also includes optimal control, topological derivative
Fireshape [18]	AD + automated adjoint (pyadjoint)
Gangl et al. [14]	Shape Hessians, surface PDEs, time-dependent PDEs
FEMorph [63]	Shape Hessians
FEniTop [32]	Parallel SIMP code
Aage et al. [31]	parallel MMA optimizer, designed for HPC scalability
Null Space Optimizer [19]	Minimal parameter tuning, handles many constraints via null-space gradient flow

Table 4: Comparison of open-source toolboxes for shape and topology optimization.

unfitted finite element methods, including CutFEM [42], XFEM [43], ϕ-FEM [44], and the unfitted FE approach of [45] would enable a more accurate approximation of the shape derivative and will be investigated in future work. In particular, ϕ-FEM [44] and the method proposed in [45] are both based on a level set representation of the geometry, thus appear especially well suited for extending the framework developed in the present work. The approach of [45] is based on the Hadamard formulation (4) of the shape derivative, whereas `FormOpt` relies on the distributed formulation (5). Consequently, incorporating this method would require the ability to compute integrals over the interface $\phi = 0$, which entails additional implementation effort, particularly with respect to efficient parallelization within the `FEniCSx` framework. Such an extension would also enable the treatment of boundary shape functionals, which are of significant practical interest. Other relevant extensions include the treatment of time-dependent problems, the use of topological derivatives [22, 21] and the integration of automatic differentiation tools [61, 33, 14, 64, 18] to facilitate the computation of adjoints and shape derivatives.

CRediT authorship contribution statement

Josué D. Díaz-Avalos: Writing – review & editing, Writing – original draft, Software, Supervision, Methodology, Investigation, Formal analysis, Conceptualization

Antoine Laurain: Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization

Replication of results

The authors state that all the data necessary to replicate the results are presented in the manuscript.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code used to generate the numerical results presented in this paper is publicly available at

`https://github.com/JD26/FormOpt.git`

The repository includes the Jupyter notebook `code/Examples.ipynb`, which invokes the relevant routines and contains the numerical tests reported in the paper.

Appendix A: implementation details

A.1: Toolbox structure

The main classes included in `formopt` are the following:

- `Model`: This is an abstract class that serves as a base for user-defined classes (i.e., subclasses of `Model`). It must contain the weak formulations of the state and adjoint equations, the cost functional, the constraints, the distributed shape derivative components, as well as the bilinear form used to compute the velocity θ, all written exclusively using Unified Form Language (UFL) and native Python functions. To achieve this, the following methods must be overridden:

```
pde(level_set_func)
adjoint(level_set_func, states)
cost(level_set_func, states)
constraint(level_set_func, states)
derivative(level_set_func, states, adjoints)
bilinear_form(velocity_func, test_func)
```

  The `Model` class provides its subclasses with methods to create the initial level set function ϕ^0 and to run Algorithm 1 using different parallelism modes:

```
create_initial_level(centers, radii) # create the initial level set function
runDP(...) # data parallelism
runTP(...) # task parallelism
runMP(...) # mixed parallelism
```

  These methods must **not** be overridden.

- `Velocity`: This class builds and solves the velocity equation (29). Since the bilinear form B remains unchanged throughout the iterations, the `__init__` method of this class compiles the left-hand side of the linear system corresponding to (29). Thus, at each iteration, only the right-hand side is updated before solving the linear system; see the `run` method of this class.

- `Level`: This class provides the level set function at each iteration of Algorithm 1 by solving the transport equation (31). The linear system is precompiled during initialization; see its `__init__` method. Calling the `run` method updates the level set function. Before starting the time-stepping iterations to solve the discretized version of (31) on a user-defined time interval according to the iterative scheme (37), the right-hand side is compiled, while only the left-hand side is updated during these iterations.

- `InitialLevel`: This class creates the initial level set function ϕ^0 from a set of centers and radii. Its `func` method returns a callable function representing ϕ^0.

- `PPL`: This class implements the Lagrangian method developed in [50] to handle the constraints; see Section 6.4.

- `AdapTime`: This class implements an adaptive time-stepping method to estimate the number of steps and the final time to solve the transport equation (31).

- `Reinit`: This class implements the reinitialization of the level set function ϕ^i by approximating the distance function associated to $\Omega^i = \{x \in \mathcal{D} \mid \phi^i(x) < 0\}$.

The following are some helper functions:

- `create_domain_2d_<DP|TP|MP>`: Here, `DP|TP|MP` refers to the parallelism paradigm. This function uses `Gmsh` to create 2D polygonal domains from a set of vertices, along with marked boundaries. Interior holes and curves (formed by line segments) are supported.

- `create_space`: This is a wrapper function for creating a `FEniCSx` function space.

- `homogeneous_dirichlet`: This function creates homogeneous Dirichlet boundary conditions on faces of dimension $d - 1$.

- `dir_extension_from`: This function computes the Dirichlet extension of the solutions corresponding to a set of partial differential equations. See "Dirichlet extension" in Subsection 6.7 for the mathematical details,

The `runDP`, `runTP`, and `runMP` functions implement Algorithm 1 using different parallelism modes. The corresponding parameters, together with their types and default values, are:

```
niter: int = 100,
dfactor: float = 1e-2,
lv_time: Tuple[float, float] = (1e-3, 1e-1),
lv_iter: Tuple[int, int] = (8, 16),
smooth: bool = False,
reinit_step: int | bool = False,
reinit_pars: Tuple[int, float] = (8, 1e-1),
start_to_check: int = 30,
ctrn_tol: float = 1e-2,
lgrn_tol: float = 1e-2,
cost_tol: float = 1e-2,
prev: int = 10,
random_pars: Tuple[int, float] = (1, 0.05)
```

The `niter` parameter is the number of iterations. The parameters `dfactor`, `lv_time`, `lv_iter`, and `smooth` are related to the level set equation. See Subsection 6.6 and "Adaptive time-stepping" in Subsection 6.7 for more details. The parameters `reinit_step` and `reinit_pars` are related to the reinitialization of the level set function. See "Reinitialization" in Subsection 6.7 for more details. The parameters `start_to_check`, `ctrn_tol`, `lgrn_tol`, `cost_tol`, and `prev`, are related to the errors and tolerances to decide when to stop the algorithm. See "Stopping criterion" in Subsection 6.7. Finally, the `random_pars` parameter adds randomness to the integration of the level set equation. See the last part of "Adaptive time-stepping" in Subsection 6.7 for more details. The `runMP` function has an additional parameter:

```
sub_comm: MPI.Comm # Without default value
```

It is a MPI communicator obtained by splitting the processes in groups and it must be created when the modality is mixed parallelism. We point out that the user-defined subclasses inherit the `runDP`, `runTP`, `runMP` functions from the `Model` class. See Subsection 6.3 for a description of the parallelism paradigms.

Several classes and functions are used internally, and the user does not need to understand how to use them. Below, we describe the stages the user must follow, along with the classes and functions that must be used:

1. Creation of a model class (a subclass of `Model`) containing the problem equations.
2. Definition of the domain $\mathcal{D}$, function space, and boundary conditions. To facilitate this step, we have provided the functions

   ```
   create_domain_2d_DP(...) # for data parallelism
   create_domain_2d_TP(...) # for task parallelism
   create_domain_2d_MP(...) # for mixed parallelism
   ```

   and several other helper functions for setting Dirichlet and Neumann boundary conditions.
3. Initialization and execution of the model by calling the method

   ```
   runDP(...) # for data parallelism
   runTP(...) # for task parallelism
   runMP(...) # for mixed parallelism
   ```

Only in the last two stages does the user need to consider the parallelism paradigm. Since the problem model is a subclass of `Model`, no parallelism considerations are needed during its creation.

All the tests presented in this work are contained in `test.py`. For each test, there is a corresponding folder in `results/`, where the results are saved. In `code/Tutorial.ipynb`, a tutorial covering a few of these tests can be found. The files `load.py` and `plots.py` contain auxiliary code for visualizing the results. We recommend ParaView [65] to view the results which are saved in XDMF format. For further details, documentation is available at `JD26.github.io/FormOpt/`.

A.2: Model construction

The first step of a model construction is to create a subclass of `Model` with the following required attributes and methods:

```
from formopt import Model

class MyModel(Model):

    def __init__(self, dim, domain, space, path):
        self.dim = dim
        self.domain = domain
        self.space = space
        self.path = path

    def pde(self, phi):
        pass

    def adjoint(self, phi, U):
        pass

    def cost(self, phi, U):
        pass

    def constraint(self, phi, U):
        pass

    def derivative(self, phi, U, P):
        pass

    def bilinear_form(self, th, xi):
        pass
```

The attributes `dim`, `domain`, `space`, and `path` correspond, respectively, to the dimension d of the problem domain $\mathcal{D}$, the mesh that represents $\mathcal{D}$, the function space for the solutions of the state and adjoint equations (we assume the same space for both), and the path where the results will be saved.

The function arguments `phi`, `U`, `P` represent the level set function, the state solutions, and the adjoint solutions, respectively. The `pde` method defines the partial differential equations of the problem. It must return a list with elements of the form `(wk, bc)`, where `wk` is the weak formulation of the equations, and `bc` is a list of Dirichlet boundary conditions. The `adjoint` method defines the adjoint equations of the problem and must return a list with the same structure as that of `pde`. The `cost` method must return the cost functional $J(\Omega)$. The `constraint` method must return a list with the components of $C(\Omega)$. The `derivative` method must return two tuples, each containing the derivative components S_0 and S_1, corresponding to the cost functional and the constraints. Finally, the `bilinear_form` method must return the chosen bilinear form B, and `th`, `xi` represent the arguments θ, ξ in $B(\theta, \xi)$.

We adopt the convention that all components of this class are implemented exclusively using UFL and native Python functions. For example, the following UFL functions were used across all our models (see `models.py`):

```
from ufl import (
    TrialFunction, TestFunction,
    FacetNormal, Identity, Measure,
    SpatialCoordinate, Coefficient,
    conditional, indices, as_vector,
    inner, outer, grad, sym, dot,
    lt, pi, cos, sqrt, nabla_div
)
```

and the following `bilinear_form` method was used in Examples 3 and 4:

```
def bilinear_form(self, th, xi):
    nv = FacetNormal(self.domain)
    B = 0.1 * dot(th, xi) * self.dx
    B += inner(grad(th), grad(xi)) * self.dx
    B += 1e4 * dot(th, nv) * dot(xi, nv) * self.ds
```

```
    for sb in self.sub:
        B += 1e4 * sb * dot(th, xi) * self.dx
    return B, False
```

Details about the input and outputs of the required methods can be found in the documentation of the `Model` class.

A.3: Parallelization

We implement parallelism using the Message Passing Interface (MPI). Communication between processes is handled through a *communicator*, which must be specified at the start of the program. The communicator is used in particular to broadcast data from one process to all other processes or to gather values distributed across multiple processes into a single process. We use the default communicator `MPI.Comm_World`. The model problem (i.e., a subclass of `Model`) and its initialization are independent of the parallelism paradigm. However, when creating the domain, the appropriate communicator must be passed. For all paradigms, we begin by accessing the MPI communicator (`comm`), the number of processes (`size`), and the current process (`rank`):

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
size = comm.size
rank = comm.rank
```

To create the domain for data parallelism, use `comm`, the main communicator that connect all the processes. For 2D domains, we provide the function `create_domain_2d_DP`, which internally uses `comm`. This function can be imported from `formopt`.

Before starting task parallelism, we check that the number of processes matches the number of state equations. We use the variable `task_nbr` to store this number. After this verification, call the "self" communicator (used for communication within a single process):

```
if size != task_nbr:
    return
comm_self = MPI.COMM_SELF
```

Then use `comm_self` to create the domain in each process. Alternatively, one can import the function `create_domain_2d_TP` to create 2D domains. It is not necessary to pass `comm_self` to this function. Note that if adjoint equations are present, their number is assumed to be equal to the number of state equations. This assumption is particularly important in task parallelism, where one process is assigned to each state equation, and after they are solved, the same applies to the adjoint equations.

In the mixed parallelism paradigm (MP), processes are divided into groups of equal size. Thus, the total number of processes (`size`) must be divisible by the number of groups (`nbr_groups`), where `nbr_groups` corresponds to the number of state equations (and adjoint equations, if present). Once this condition is verified, we assign a `color` to each process: processes in the same group share the same `color`. We expect each group to contain more than one process; otherwise, the configuration corresponds to task parallelism. For simplicity, we do not consider the case where `size` is not divisible by `nbr_groups`, thus avoiding groups of different sizes.

The `Split` function returns the sub-communicator associated to each group of processes:

```
if size%nbr_groups != 0:
    return
color = rank * nbr_groups // size
sub_comm = comm.Split(color, rank)
```

Using `sub_comm`, one copy of the same domain must be created in each group of processes. Alternatively, one can use the function `create_domain_2d_MP` to do this, passing `sub_comm` as an argument.

Appendix B: implementation details for the inverse elasticity example

We provide implementation details for the inverse elasticity problem of Section 7.1. The model class for this problem is called `InverseElasticity`:

```
from formopt import Model

class InverseElasticity(Model):
    def __init__(self, dim, domain, space, path):
        pass
```

In addition to the mandatory parameters `dim`, `domain`, `space`, and `path`, the initializer includes the following parameters:

```
def __init__(self,dim,domain,space,forces,ds_forces,ds1,dirbc_partial,dirbc_total,path):
```

The parameters `forces` and `ds_forces` are lists containing the forces and their corresponding surfaces of application, respectively. The parameter `ds1` corresponds to the surface Γ_1. Here, we assume that $f \equiv \mathbf{0}$ on a subset of Γ_1 in (9). Accordingly, the `ds_forces` list contains only those portions of Γ_1 on which f is not zero. The parameters `dirbc_partial` and `dirbc_total` are the homogeneous Dirichlet conditions imposed on Γ_0 and Γ, respectively.

In `__init__` are defined the variables and functions that will be used to write the problem equations. For instance, the functions σ, ϵ, and A_Ω are defined as Python lambda functions:

```
self.sigma = lambda w: lm * nabla_div(w) * self.Id + 2.0 * mu * self.epsilon(w)
self.epsilon = lambda w: sym(grad(w))
self.A = lambda w: conditional(lt(w, 0.0), 10.0, 1.0)
```

Here, `self.A` is an attribute of the `InverseElasticity` class and represents A_Ω with $\kappa = 10$, see (11). The argument passed to `self.A` will be the level set function: if negative, `self.A` returns 10, else 1.

The weak formulations for the force application (12) and the observed displacement (13), along with their boundary conditions, are returned by the `f_prob` and `g_prob` methods:

```
def f_prob(self, u, w, phi, f, df):
    # f-problem
    su, ew = self.sigma(u), self.epsilon(w)
    W = self.A(phi) * inner(su, ew) * self.dx - dot(f, w) * df
    return (W, self.bcF)

def g_prob(self, v, w, phi, g):
    # g-problem
    sv, sg, ew = self.sigma(v), self.sigma(g), self.epsilon(w)
    W = self.A(phi) * inner(sv, ew) * self.dx + self.A(phi) * inner(sg, ew) * self.dx
    return (W, self.bcG)
```

Using these methods we write the required `pde` method. It returns a list (`F+G`) with the weak formulation and boundary condition corresponding to each applied force and boundary displacement:

```
def pde(self, phi):
    a, b = TrialFunction(self.space), TestFunction(self.space)
    zipf = zip(self.fs, self.dfs)
    F = [self.f_prob(a, b, phi, f, df) for f, df in zipf]
    G = [self.g_prob(a, b, phi, g) for g in self.gs]
    return F + G
```

Similarly, we write the `adj_f_prob` and `adj_g_prob` methods to get the adjoint problems, and then we call them to collect all the adjoint equations in the required `adjoint` method:

```
def adjoint(self, phi, U):
    a, b = TrialFunction(self.space), TestFunction(self.space)
    AF, AG = [], []
    for u, v, g in zip(U[:self.N], U[self.N:], self.gs):
        AF += [self.adj_f_prob(a, b, phi, u, v, g)]
        AG += [self.adj_g_prob(a, b, phi, u, v, g)]
    return AF + AG
```

Observe that in the `adjoint` method we iterate over `U`: the sub-lists `U[:self.N]` and `U[self.N:]` contain

the solutions of (9) and (10), respectively. The `AF` list containts the adjoint equations corresponding to the problems with force application, while the `AG` list containts the adjoint equations corresponding to the problems with boundary displacement.

The cost functional (8) is returned by the `cost` method:

```
def cost(self, phi, U):
    uvg = zip(U[:self.N], U[self.N:], self.gs)
    ug = zip(U[:self.N], self.gs)
    Ja = [dot(u - v - g, u - v - g) * self.dx for u, v, g in uvg]
    Jb = [dot(u - g, u - g) * self.ds1 for u, g in ug]
    return 0.5 * (self.alpha * sum(Ja) + self.beta * sum(Jb))
```

The weights `self.alpha` and `self.beta` are defined in `__init__`.

Since there are no constraints in this problem, the `constraint` method must return an empty list:

```
def constraint(self, phi, U):
    return []
```

In order to write the `derivative` method, we first write the derivative components S_0 and S_1 separately:

```
def S0(self, u, v, q, g, phi):
    i, j, k = indices(3)
    sq, eg = self.sigma(q), self.epsilon(g)
    s0a = grad(g).T * (u - v - g)
    s0b = as_vector(sq[i,j] * (grad(eg))[i,j,k], (k))
    return -self.alpha * s0a + self.A(phi) * s0b
```

and

```
def S1(self, u, v, p, q, g, phi):
    su, sp, sq = self.sigma(u), self.sigma(p), self.sigma(q)
    svg = self.sigma(v + g)
    ep, eq = self.epsilon(p), self.epsilon(q)
    uvg = u - v - g
    s1i = inner(su, ep) + inner(svg, eq)
    s1i = self.alpha * dot(uvg, uvg) / 2.0 + self.A(phi) * s1i
    s1j = grad(u).T * sp + grad(p).T * su + grad(v).T * sq + grad(q).T * svg
    return s1i * self.Id - self.A(phi) * s1j
```

The `derivative` method collects the derivative components for each pair of force and displacement by evaluating the `S0` and `S1` methods at state and adjoint solutions, then the sums `sum(S0)` and `sum(S1)` are returned:

```
def derivative(self, phi, U, P):
    fu = U[:self.N]
    gv = U[self.N:]
    fp = P[:self.N]
    gq = P[self.N:]
    uvqg = zip(fu, gv, gq, self.gs)
    uvpqg = zip(fu, gv, fp, gq, self.gs)
    S0_list = [self.S0(u, v, q, g, phi) for u, v, q, g in uvqg]
    S1_list = [self.S1(u, v, p, q, g, phi) for u, v, p, q, g in uvpqg]
    return (sum(S0_list), []), (sum(S1_list), [])
```

Empty lists are returned in the place corresponding to the derivative components of the constraints.

Finally, we write the bilinear form used to compute the velocity field θ by solving (35):

```
def bilinear_form(self, th, xi):
    biform = 0.1 * dot(th, xi) * self.dx + inner(grad(th), grad(xi)) * self.dx
    return biform, True
```

By returning `True` we are specifying homogeneous Dirichlet boundary condition, that is, $\mathbb{H} = H_0^1(\Omega)$.

In the numerical experiments, results using all parallelism modes are reported, but here we describe only

the test using mixed parallelism. We start by calling the MPI communicator:

```
comm = MPI.COMM_WORLD
rank = comm.rank
size = comm.size
```

In the two inverse elasticity examples, we consider three and eight pairs of applied forces and their corresponding displacement observations, respectively. Thus, there are six (resp. sixteen) state problems. Setting `nbr_groups = 6` (resp. `16`), and considering `size=12` processes (resp. `32`), the number of processes `size` is a multiple of `nbr_groups`, and each group contains two processes. Then, the sub-communicator can be created:

```
color = rank * nbr_groups // size
sub_comm = comm.Split(color, rank)
```

Both examples have similar structure; the main differences lie in the number of inclusions and the number of force–displacement pairs. We now describe the examples, alternating between them.

We first specify the output path for saving results, the domain dimension ($d = 2$), the rank (i.e., the number of components of the state/adjoint solutions), and the mesh-size parameter that will be used by `Gmsh` to create the mesh:

```
test_path = Path("../results/t08/") #It corresponds to the first example
dim = 2
rank_dim = 2
mesh_size = 0.015
```

The vertices and the boundary parts of the truncated semi-ellipse $\mathcal{D}$ for the second example are defined as follows:

```
npts = 80
part = npts // 8

xycoords = semi_ellipse(0.75, 0.5, 0.15, npts)
vertices=np.column_stack(xycoords)

# 1 dirichlet boundary and 8 neumann boundaries
dir_idx, dir_mkr = [npts], 1
neu_idxA,neu_mkrA = np.arange(1, part + 1), 2
neu_idxB,neu_mkrB = np.arange(part + 1, 2 * part + 1), 3
neu_idxC,neu_mkrC = np.arange(2 * part + 1,3 * part + 1), 4
neu_idxD,neu_mkrD = np.arange(3 * part + 1,4 * part + 1), 5
neu_idxE,neu_mkrE = np.arange(4 * part + 1,5 * part + 1), 6
neu_idxF,neu_mkrF = np.arange(5 * part + 1,6 * part + 1), 7
neu_idxG,neu_mkrG = np.arange(6 * part + 1,7 * part + 1), 8
neu_idxH,neu_mkrH = np.arange(7 * part + 1, npts), 9

boundary_parts = [
    (dir_idx, dir_mkr, "dir"),
    (neu_idxA, neu_mkrA, "neuA"),
    (neu_idxB, neu_mkrB, "neuB"),
    (neu_idxC, neu_mkrC, "neuC"),
    (neu_idxD, neu_mkrD, "neuD"),
    (neu_idxE, neu_mkrE, "neuE"),
    (neu_idxF, neu_mkrF, "neuF"),
    (neu_idxG, neu_mkrG, "neuG"),
    (neu_idxH, neu_mkrH, "neuH"),
]
```

These variables and the sub-communicator are passed to the `create_domain_2d_MP` function:

```
output = fop.create_domain_2d_MP(
    sub_comm, color, vertices, boundary_parts, mesh_size, path=test_path, plot=True
)
domain, nbr_tri, boundary_tags = output
```

By default, `plot=False`, which means that the mesh generated by `Gmsh` is not plotted.

In both examples, there is only one subset of Γ on which zero displacement is imposed. In addition, it is necessary to consider the boundary condition of problem (13). To this end, we create the function space and the Dirichlet boundary conditions using predefined functions from our module:

```
space = fop.create_space(domain, "CG", rank_dim)
dirbc_partial = fop.homogeneous_dirichlet(domain, space, boundary_tags, [dir_mkr], rank_dim)
dirbc_total = fop.homogeneous_boundary(domain, space, dim, rank_dim)
```

Boundary measures are created to impose the Neumann conditions, i.e., the forces applied along the boundary Γ_1:

```
ds_parts = fop.marked_ds(domain, boundary_tags, [bR_mkr,neu_mkrA,neu_mkrB,neu_mkrC,bL_mkr])
ds_forces = [ds_parts[1], ds_parts[2], ds_parts[3]]
ds1 = sum(ds_parts[1:], start = ds_parts[0])
```

The above snippet corresponds to the first example, where three forces are applied on three different parts of Γ_1. The `ds_forces` list contains the boundary parts where each force is applied, and `ds1` groups these parts together. The markers `bR_mkr` and `bL_mkr` correspond to segments of Γ_1 where no forces are applied; they are located at the right-bottom and left-bottom parts of Γ_1, respectively. The markers `neu_mkrA`, `neu_mkrB`, and `neu_mkrC` correspond to segments where the forces are applied. In the second example, the applied forces cover the entire boundary Γ_1.

Finally, we pass all these variables to an object of the `InverseElasticity` model:

```
md = InverseElasticity(
    dim, domain, space, forces, ds_forces, ds1, dirbc_partial, dirbc_total, test_path
)
```

Before running the examples, we define a level set function to set the initial inclusions in $\mathcal{D}$. For the second example (see Figure 4), two balls of radius 0.15 are considered as initial inclusions:

```
centers = np.array([(-0.3, 0.4), (0.3, 0.4)])
radii = np.array([0.15, 0.15])
md.create_initial_level(centers, radii, factor=-1.0)
```

Notice that we set `factor=-1.0` to have the initial domain Ω^0 as the union of the balls; see Subsection 6.7 for more details.

References

[1] M. C. Delfour, J.-P. Zolésio, Shapes and geometries, 2nd Edition, Vol. 22 of Advances in Design and Control, Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2011, metrics, analysis, differential calculus, and optimization. doi:10.1137/1.9780898719826.

[2] J. Sokołowski, J.-P. Zolésio, Introduction to shape optimization, Vol. 16 of Springer Series in Computational Mathematics, Springer, Berlin, 1992, shape sensitivity analysis. doi:10.1007/978-3-642-58106-9.

[3] M. P. Bendsøe, O. Sigmund, Topology optimization, Springer, Berlin, 2003, theory, methods and applications.

[4] O. Sigmund, A 99 line topology optimization code written in matlab, Struct. Multidiscip. Optim. 21 (2) (2014) 120–127. doi:10.1007/s001580050176.

[5] S. Osher, J. A. Sethian, Fronts propagating with curvature-dependent speed: algorithms based on Hamilton-Jacobi formulations, J. Comput. Phys. 79 (1) (1988) 12–49. doi:10.1016/0021-9991(88)90002-2.

[6] M. Y. Wang, S. Zhou, Phase field: a variational method for structural topology optimization, CMES Comput. Model. Eng. Sci. 6 (6) (2004) 547–566. doi:10.3970/cmes.2004.006.547.

[7] S. Amstutz, H. Andrä, A new algorithm for topology optimization using a level-set method, J. Comput. Phys. 216 (2) (2006) 573–588. doi:10.1016/j.jcp.2005.12.015.

[8] G. Allaire, F. Jouve, A.-M. Toader, A level-set method for shape optimization, C. R. Math. Acad. Sci. Paris 334 (12) (2002) 1125–1130. doi:10.1016/S1631-073X(02)02412-3.

[9] G. Allaire, F. Jouve, A.-M. Toader, Structural optimization using sensitivity analysis and a level-set method, J. Comput. Phys. 194 (1) (2004) 363–393. doi:10.1016/j.jcp.2003.09.032.

[10] A. L. Gain, G. H. Paulino, A critical comparative assessment of differential equation-driven methods for structural topology optimization, Structural and Multidisciplinary Optimization 48 (4) (2013) 685–710. doi:10.1007/s00158-013-0935-4.

[11] N. P. van Dijk, K. Maute, M. Langelaar, F. van Keulen, Level-set methods for structural topology optimization: a review, Struct. Multidiscip. Optim. 48 (3) (2013) 437–472. doi:10.1007/s00158-013-0912-y.

[12] E. Andreassen, A. Clausen, M. Schevenels, B. S. Lazarov, O. Sigmund, Efficient topology optimization in matlab using 88 lines of code, Structural and Multidisciplinary Optimization 43 (1) (2010) 1–16. doi:10.1007/s00158-010-0594-7.
URL http://dx.doi.org/10.1007/s00158-010-0594-7

[13] G. Allaire, O. Pantz, Structural optimization with FreeFem++, Struct. Multidiscip. Optim. 32 (3) (2006) 173–181. doi:10.1007/s00158-006-0017-y.

[14] P. Gangl, K. Sturm, M. Neunteufel, J. Schöberl, Fully and semi-automated shape differentiation in ngsolve, Struct. Multidiscip. Optim. 63 (3) (2021) 1579–1607. doi:10.1007/s00158-020-02742-w.

[15] I. A. Baratta, J. P. Dean, J. S. Dokken, M. Habera, J. S. Hale, C. N. Richardson, M. E. Rognes, M. W. Scroggs, N. Sime, G. N. Wells, DOLFINx: the next generation FEniCS problem solving environment, preprint (2023). doi:10.5281/zenodo.10447666.

[16] S. Blauth, cashocs: A computational, adjoint-based shape optimization and optimal control software, SoftwareX 13 (2021) 100646. doi:10.1016/j.softx.2020.100646.

[17] A. Laurain, A level set-based structural optimization code using FEniCS, Structural and Multidisciplinary Optimization 58 (3) (2018) 1311–1334. doi:10.1007/s00158-018-1950-2.

[18] A. Paganini, F. Wechsung, Fireshape: a shape optimization toolbox for Firedrake, Struct. Multidiscip. Optim. 63 (5) (2021) 2553–2569. doi:10.1007/s00158-020-02813-y.

[19] F. Feppon, Density-based topology optimization with the null space optimizer: a tutorial and a comparison, Struct. Multidiscip. Optim. 67 (1) (2024) Paper No. 4, 34. doi:10.1007/s00158-023-03710-w.

[20] M. Y. Wang, X. Wang, D. Guo, A level set method for structural topology optimization, Comput. Methods Appl. Mech. Engrg. 192 (1-2) (2003) 227–246. doi:10.1016/S0045-7825(02)00559-5.

[21] A. A. Novotny, J. Sokołowski, Topological derivatives in shape optimization, Interaction of Mechanics and Mathematics, Springer, Heidelberg, 2013. doi:10.1007/978-3-642-35245-4.

[22] J. M. M. L. Filho, A. T. A. Gomes, A. A. Novotny, A parallel FreeFEM framework for topology optimization of structures into three spatial dimensions, Finite Elem. Anal. Des. 250 (2025) Paper No. 104384. doi:10.1016/j.finel.2025.104384.

[23] V. J. Challis, A discrete level-set topology optimization code written in matlab, Struct. Multidiscip. Optim. 41 (3) (2009) 453–464. doi:10.1007/s00158-009-0430-0.

[24] P. Fulmański, A. Laurain, J.-F. Scheid, J. Sokołowski, Level set method with topological derivatives in shape optimization, Int. J. Comput. Math. 85 (10) (2008) 1491–1514. doi:10.1080/00207160802033350.

[25] F. de Gournay, Velocity extension for the level-set method and multiple eigenvalues in shape optimization, SIAM J. Control Optim. 45 (1) (2006) 343–367. doi:10.1137/050624108.
URL https://doi.org/10.1137/050624108

[26] A. Laurain, Distributed and boundary expressions of first and second order shape derivatives in nonsmooth domains, Journal de Mathématiques Pures et Appliquées 134 (2020) 328–368. doi:10.1016/j.matpur.2019.09.002.

[27] A. Laurain, K. Sturm, Distributed shape derivative *via* averaged adjoint method and applications, ESAIM. Mathematical Modelling and Numerical Analysis 50 (4) (2016) 1241–1267. doi:10.1051/m2an/2015075.

[28] M. Berggren, A unified discrete-continuous sensitivity analysis method for shape optimization, in: Applied and numerical partial differential equations, Vol. 15 of Comput. Methods Appl. Sci., Springer, New York, 2010, pp. 25–39. doi:10.1007/978-90-481-3239-3_4.

[29] R. Hiptmair, A. Paganini, S. Sargheini, Comparison of approximate shape gradients, BIT 55 (2) (2015) 459–485. doi:10.1007/s10543-014-0515-z.

[30] Z. J. Wegert, J. Manyer, C. N. Mallon, S. Badia, V. J. Challis, Gridaptopopt.jl: a scalable julia toolbox for level set-based topology optimisation, Structural and Multidisciplinary Optimization 68 (1) (Jan. 2025). doi:10.1007/s00158-024-03927-3.

[31] N. Aage, E. Andreassen, B. S. Lazarov, Topology optimization using PETSc: an easy-to-use, fully parallel, open source topology optimization framework, Struct. Multidiscip. Optim. 51 (3) (2015) 565–572. doi:10.1007/s00158-014-1157-0.

[32] Y. Jia, C. Wang, X. S. Zhang, Fenitop: a simple fenicsx implementation for 2d and 3d topology optimization supporting parallel computing, Structural and Multidisciplinary Optimization 67 (8) (2024). doi:10.1007/s00158-024-03818-7.

[33] S. Blauth, Version 2.0 - cashocs: A computational, adjoint-based shape optimization and optimal control software, SoftwareX 24 (2023) 101577. doi:10.1016/j.softx.2023.101577.

[34] Y. F. Albuquerque, A. Laurain, I. Yousept, Level set-based shape optimization approach for sharp-interface reconstructions in time-domain full waveform inversion, SIAM J. Appl. Math. 81 (3) (2021) 939–964. doi:10.1137/20M1378090.

[35] A. Morassi, E. Rosset, Uniqueness and stability in determining a rigid inclusion in an elastic body, Mem. Amer. Math. Soc. 200 (938) (2009) viii+58. doi:10.1090/memo/0938.

[36] H. Meftahi, J.-P. Zolésio, Sensitivity analysis for some inverse problems in linear elasticity via minimax differentiability, Applied Mathematical Modelling 39 (5) (2015) 1554–1576. doi:10.1016/j.apm.2014.09.026.

[37] A. Henrot, M. Pierre, Shape variation and optimization, Vol. 28 of EMS Tracts in Mathematics, European Mathematical Society (EMS), Zürich, 2018, a geometrical analysis, English version of the French publication [MR2512810] with additions and updates. doi:10.4171/178.
URL https://doi.org/10.4171/178

[38] A. Laurain, P. T. P. Lopes, J. C. Nakasato, An abstract Lagrangian framework for computing shape derivatives, ESAIM. Control, Optimisation and Calculus of Variations 29 (2023) article 5. doi:10.1051/cocv/2022078.

[39] A. Laurain, P. T. P. Lopes, On second-order tensor representation of derivatives in shape optimization, Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences 382 (2277) (2024) 20230300. doi:10.1098/rsta.2023.0300.

[40] D. D. Ang, D. D. Trong, M. Yamamoto, Identification of cavities inside two-dimensional heterogeneous isotropic elastic bodies, J. Elasticity 56 (3) (1999) 199–212. doi:10.1023/A:1007661505879.

[41] R. Kohn, M. Vogelius, Determining conductivity by boundary measurements, Comm. Pure Appl. Math. 37 (3) (1984) 289–298. doi:10.1002/cpa.3160370302.

[42] E. Burman, S. Claus, P. Hansbo, M. G. Larson, A. Massing, CutFEM: discretizing geometry and partial differential equations, Internat. J. Numer. Methods Engrg. 104 (7) (2015) 472–501. doi:10.1002/nme.4823.

[43] T. Belytschko, N. Moës, S. Usui, S. Parimi, Arbitrary discontinuities in finite elements, International Journal for Numerical Methods in Engineering 50 (4) (2001) 993–1013. doi:10.1002/1097-0207(20010210)50:4<993::AID-NME164>3.0.CO;2-M.

[44] M. Duprez, A. Lozinski, ϕ-fem: A finite element method on domains defined by level-sets, SIAM Journal on Numerical Analysis 58 (2) (2020) 1008–1028. arXiv:https://doi.org/10.1137/19M1248947, doi:10.1137/19M1248947.
URL https://doi.org/10.1137/19M1248947

[45] J. T. Shahan, S. W. Walker, Exact shape derivatives with unfitted finite element methods, Journal of Numerical Mathematics 34 (1) (2026) 91–124 [cited 2026-03-26]. doi:doi:10.1515/jnma-2024-0113.
URL https://doi.org/10.1515/jnma-2024-0113

[46] Gross, Sven, Olshanskii, Maxim A., Reusken, Arnold, A trace finite element method for a class of coupled bulk-interface transport problems, ESAIM: M2AN 49 (5) (2015) 1303–1330. doi:10.1051/m2an/2015013.
URL https://doi.org/10.1051/m2an/2015013

[47] M. Burger, A framework for the construction of level set methods for shape optimization and reconstruction, Interfaces Free Bound. 5 (3) (2003) 301–329. doi:10.4171/IFB/81.
URL https://doi.org/10.4171/IFB/81

[48] M. Eigel, K. Sturm, Reproducing kernel hilbert spaces and variable metric algorithms in pde-constrained shape optimization, Optimization Methods and Software 33 (2) (2018) 268–296. arXiv:https://doi.org/10.1080/10556788.2017.1314471, doi:10.1080/10556788.2017.1314471.
URL https://doi.org/10.1080/10556788.2017.1314471

[49] A. Toselli, O. Widlund, Domain decomposition methods—algorithms and theory, Vol. 34 of Springer Series in Computational Mathematics, Springer-Verlag, Berlin, 2005. doi:10.1007/b137868.

[50] J. G. Kim, A new Lagrangian-based first-order method for nonconvex constrained optimization, Oper. Res. Lett. 51 (3) (2023) 357–363. doi:10.1016/j.orl.2023.04.006.

[51] T. Yamada, K. Izui, S. Nishiwaki, A. Takezawa, A topology optimization method based on the level set method incorporating a fictitious interface energy, Computer Methods in Applied Mechanics and Engineering 199 (45) (2010) 2876–2891. doi:https://doi.org/10.1016/j.cma.2010.05.013.
URL https://www.sciencedirect.com/science/article/pii/S0045782510001623

[52] M. G. Crandall, L. C. Evans, P.-L. Lions, Some properties of viscosity solutions of Hamilton-Jacobi equations, Trans. Amer. Math. Soc. 282 (2) (1984) 487–502. doi:10.2307/1999247.
URL https://doi.org/10.2307/1999247

[53] A. Laurain, Analyzing smooth and singular domain perturbations in level set methods, SIAM J. Math. Anal. 50 (4) (2018) 4327–4370. doi:10.1137/17M1118956.
URL https://doi.org/10.1137/17M1118956

[54] T. J. Barth, J. A. Sethian, Numerical schemes for the Hamilton-Jacobi and level set equations on triangulated domains, J. Comput. Phys. 145 (1) (1998) 1–40. doi:10.1006/jcph.1998.6007.

[55] S. Kambampati, Z. Du, H. Chung, H. A. Kim, C. Jauregui, S. Townsend, R. Picelli, X.-Y. Zhou, L. Hedges, Openlsto: Open-source software for level set topology optimization, in: 2018 Multidisciplinary Analysis and Optimization Conference, American Institute of Aeronautics and Astronautics, 2018. doi:10.2514/6.2018-3882.

[56] X. Mo, H. Zhi, Y. Xiao, H. Hua, L. He, Topology optimization of cooling plates for battery thermal management, International Journal of Heat and Mass Transfer 178 (2021) 121612. doi:10.1016/j.ijheatmasstransfer.2021.121612.

[57] T. Gao, W. Zhang, J. Zhu, Y. Xu, D. Bassir, Topology optimization of heat conduction problem involving design-dependent heat load effect, Finite Elements in Analysis and Design 44 (14) (2008) 805–813. doi:10.1016/j.finel.2008.06.001.

[58] M.-É. Lamarche-Gagnon, M. Molavi-Zarandi, V. Raymond, F. Ilinca, Additively manufactured conformal cooling channels through topology optimization, Structural and Multidisciplinary Optimization 67 (8) (Jul. 2024). doi:10.1007/s00158-024-03846-3.

[59] C. Zhuang, Z. Xiong, H. Ding, A level set method for topology optimization of heat conduction problem under multiple load cases, Comput. Methods Appl. Mech. Engrg. 196 (4-6) (2007) 1074–1084. doi:10.1016/j.cma.2006.08.005.

[60] J. D. Murray, Mathematical biology. I, 3rd Edition, Vol. 17 of Interdisciplinary Applied Mathematics, Springer, New York, 2002, an introduction.

[61] G. Allaire, M. H. Gfrerer, Autofreefem: automatic code generation with freefem and latex output for shape and topology optimization of non-linear multi-physics problems, Struct. Multidiscip. Optim. 67 (12) (2024). doi:10.1007/s00158-024-03917-5.

[62] H. Li, T. Yamada, P. Jolivet, K. Furuta, T. Kondoh, K. Izui, S. Nishiwaki, Full-scale 3d structural topology optimization using adaptive mesh refinement based on the level-set method, Finite Elements in Analysis and Design 194 (2021) 103561. doi:https://doi.org/10.1016/j.finel.2021.103561.
URL https://www.sciencedirect.com/science/article/pii/S0168874X21000457

[63] S. Schmidt, Weak and strong form shape hessians and their automatic generation, SIAM Journal on Scientific Computing 40 (2) (2018) C210–C233. arXiv:https://doi.org/10.1137/16M1099972, doi:10.1137/16M1099972.
URL https://doi.org/10.1137/16M1099972

[64] D. A. Ham, L. Mitchell, A. Paganini, F. Wechsung, Automated shape differentiation in the unified form language, Struct. Multidiscip. Optim. 60 (5) (2019) 1813–1820. doi:10.1007/s00158-019-02281-z.

[65] U. Ayachit, The ParaView Guide: A Parallel Visualization Application, Kitware, Inc., Clifton Park, NY, USA, 2015.